SKYWAY AIRLINES

Complete project & code study guide

Understand the system. Follow the data. Explain the code.

Java AWT + JDBC + MySQL

React demo payment website + PDFBox + Jakarta Mail

Prepared for your college presentation
Source reviewed: 20 October 2026
Presentation preparation: 21 October 2026

This guide explains the files actually present in your project. It starts with the overall working, then explains each class and important code block, and ends with viva answers and the complete numbered source.

Important: Payments are simulated. The QR opens a local demo website; it does not transfer money. Sample airline names and schedules belong to the project database.

Private email credentials and existing passenger tickets are not reproduced.

How to use this guide

Read the conceptual sections first. Then keep your editor open and match the method names and source line numbers. The appendix contains the complete project-authored Java, React, CSS, SQL and setup code, so you can study without switching between files.

Time available	Recommended reading
30 minutes	Read the project overview, end-to-end flow, transaction example, presentation script and viva questions.
1-2 hours	Read all class explanations, especially PassengerDetailsFrame, PaymentFrame, PaymentServer and MyBookingsFrame.
Detailed revision	Read the numbered source appendix alongside the explanation chapters. Trace one selected seat and one passenger through every method.

Reading rule: For every method, ask: who calls it, what values enter it, what work happens, what database rows change, and what happens if it fails.

What was checked

All 18 Java source files, the React payment source, stylesheet, SQL schemas and run/configuration files were read. Java compilation succeeded against the supplied JARs. This is a source-based explanation; a complete live MySQL, phone-payment and SMTP demonstration was not performed. The website build result is recorded in the setup chapter.

Scope of the complete source appendix

It includes all handwritten executable source and relevant setup files. Compiled .class files, external .jar libraries, node_modules, generated web output, package-lock.json, existing tickets and the older report are not handwritten application logic. The lockfile pins dependency versions and integrity data; its thousands of generated entries are not repeated.

Contents

How to use this guide 2

Contents 3

01 Project overview 4

02 End-to-end working flow 5

03 Database design and SQL 6

04 Java foundations used throughout 7

05 Startup, database and popups 8

06 Login, registration and dashboards 9

07 Managing and saving flights 10

08 Flight browsing and seat selection 11

09 How the custom seat map works 12

10 Passenger details and booking 13

11 Transactions: the key viva topic 14

12 The Java QR payment window 15

13 The Java HTTP payment server 16

14 The React payment website 18

15 Ticket PDF, email and random codes 19

16 Viewing bookings and cancellation 20

17 Setup, dependencies and demo checks 21

18 Present it to sir: a clear script 23

19 Viva questions with model answers 24

20 Limitations and accurate claims 26

Appendix Complete numbered source 27

01 Project overview

SkyWay is a desktop airline reservation management system. An administrator maintains flights and sees bookings. A customer registers, signs in, chooses a flight and seats, enters passenger details, completes a simulated payment, and receives a PDF ticket. The customer can later view or cancel one passenger ticket.

The problem it solves

A booking must connect a customer, a flight, one or more seats and passenger details. A simple list on screen is not enough: two customers might choose the same seat. This project stores the persistent truth in MySQL and uses transactions when several related records must change together.

Technology	Its exact job here
Java AWT	Desktop windows, buttons, text fields, lists, dialogs and custom seat/QR drawing.
JDBC + MySQL	Java sends SQL queries through the MySQL driver. MySQL stores users, flights, seats, bookings and demo payments.
Java HttpServer	Built-in HTTP server on port 8085 connects the phone website to the running Java program.
React + Vite	Phone-friendly demo payment page, served during development on port 5173.
ZXing	Turns the payment-page URL into a QR-code matrix.
Apache PDFBox	Creates an electronic ticket PDF under tickets/.
Jakarta Mail + Angus	Constructs and sends email with a PDF attachment through configured SMTP.

What this project is architecturally

It is an event-driven desktop application with a small HTTP integration. Most business logic and SQL are inside the screen classes. DBConnection and TicketService are shared helpers. It is not a strict MVC design: view, validation and database operations are often combined in the same class.

Persistent data versus temporary state

MySQL records survive closing the application. In contrast, selected seat indexes, typed passenger values, the current PaymentFrame pointer and Java screen objects live in memory. Selecting a green seat does not reserve it in MySQL. The actual reservation happens later in confirmBooking().

02 End-to-end working flow

```
START
  AirlineMain.main()
    -> register keyboard shortcuts
    -> PaymentServer.start() on 8085
    -> AirlineMain welcome window

ADMIN -> LoginFrame("Admin") -> AdminDashboard
           -> ManageFlightsFrame -> FlightFormFrame
           -> ViewBookingsFrame

CUSTOMER -> RegisterFrame (if new) -> LoginFrame("Customer")
  -> CustomerDashboard(customerId, name)
  -> AvailableFlightsFrame(customerId)
  -> SeatSelectionFrame(..., flightId) -> SeatMap
  -> PassengerDetailsFrame(..., seatIds, seatNos, price)
  -> PaymentFrame -> insert PENDING payment -> show QR
  -> phone: React App -> PaymentServer /success
  -> confirmBooking() -> MySQL COMMIT -> PNR
  -> TicketService.makePdf() -> sendEmail()
  -> MyBookingsFrame -> view / cancel / regenerate ticket
```

One concrete example

Suppose Rahul chooses sample flight AI101, priced at Rs. 4,500 per seat, and selects 1A and 1B. The selection screen calculates 2 x 4,500 = Rs. 9,000. It passes the database IDs of both seats, their displayed names and the customer/flight IDs into the passenger window.

Two passenger rows are entered. PaymentFrame creates a reference such as PAY4K7M2X and saves a PENDING payment. Its QR contains a URL shaped like http://<laptop-IP>:5173/?ref=PAY4K7M2X. The QR contains a website address, not passenger records or banking details.

The phone loads the payment using that reference. Clicking Payment Done calls the Java server. The server attempts to change PENDING to SUCCESS and invokes the matching payment window. confirmBooking() locks the flight, claims both seats, reduces available seats by two, inserts two booking rows with the same PNR and commits.

After the HTTP response is sent, the Java code creates the PDF and attempts email. Closing the completed payment window closes the passenger and seat windows and refreshes the flight list. My Bookings shows one list entry per passenger, even though the PNR is shared.

Three different identifiers

Identifier	Meaning
customer_id / flight_id / seat_id	Numeric database primary keys used to connect records. A seat_id is not the same as displayed seat_no such as 1A.
payment_ref	Unique identifier for one demo payment attempt. Retry creates a new attempt and a new QR.
PNR	Human-readable reservation reference shared by all passengers in one group. booking_id remains unique for each passenger row.

03 Database design and SQL

database/airline_db.sql creates airline_db and six InnoDB tables. InnoDB supports transactions and row locking, which the booking and cancellation logic depend on. The script drops existing tables before recreating them: use it for a fresh demo database, not as a harmless refresh of existing bookings.

Table	Columns and responsibility
admins	admin_id primary key; unique username; password. Holds administrator login records.
customers	customer_id; name; unique email; phone; password; created_at. Represents the person making bookings, not necessarily every passenger.
flights	flight_id; flight_no; airline; source/destination; departure/arrival; price; total_seats; available_seats. Unique flight_no + departure_time.
seats	seat_id; flight_id; seat_no; AVAILABLE/BOOKED status. Unique flight_id + seat_no prevents duplicate seat labels on one flight.
bookings	booking_id; shared pnr; customer_id; flight_id; seat_id; passenger name/age/gender; booking_date; CONFIRMED/CANCELLED; ticket_price. One row per passenger.
payments	payment_id; unique payment_ref; customer_id; flight_id; seats text; passengers count; amount; PENDING/SUCCESS/FAILED; nullable pnr; created_at.

```
customers 1 ---- many bookings many ---- 1 flights
                        |                  |
                      many                 1
                        |                  |
                        1                many
                      seats ---------------+

customers 1 ---- many payments many ---- 1 flights
PNR groups booking rows logically; there is no separate PNR table.
```

SQL ideas you should be able to explain

PRIMARY KEY uniquely identifies a row. **AUTO_INCREMENT** asks MySQL to generate the next numeric ID. **FOREIGN KEY** requires a referenced parent record to exist. **UNIQUE** rejects duplicate key values. **INDEX** helps searches; the PNR index is not a uniqueness constraint.

JOIN combines related rows so a booking can display its seat number and route. **WHERE** filters rows. **ORDER BY** controls display order. **COUNT(*)** counts matching bookings. **NOW()** compares departure with database time. **ENUM** restricts the set of stored status strings.

The flights CHECK keeps available_seats between zero and total_seats on database versions that enforce CHECK constraints. The application also updates counts transactionally. DECIMAL(10,2) stores fixed-point money in MySQL, although the Java code reads/calculates prices with double.

Cascade behavior matters

Deleting a flight cascades to its seats, bookings and payments. Deleting seats also cascades to related booking rows. The admin code blocks deletion when confirmed bookings exist, but deletion can remove cancelled booking history. Changing seat capacity with only cancelled bookings can also remove that history when seats are rebuilt.

Initial data and migration

The main SQL inserts one admin, one customer and three December 2026 sample flights, each with 12 seats (1A-1F and 2A-2F). The separate payments_table.sql only creates the payments table if missing; it does not recreate other tables or migrate an incompatible existing payments schema.

04 Java foundations used throughout

Syntax / concept	Meaning in this project
extends Frame	A screen inherits a top-level AWT window and its methods such as add(), setSize() and dispose().
implements ActionListener	The class promises to supply actionPerformed(ActionEvent), called when a registered button is activated.
this	The current object. addActionListener(this) makes this screen receive button events.
new SomeFrame(...)	Creates another screen object; its constructor builds and shows the window.
static	Belongs to the class rather than an individual object. DBConnection.getConnection() needs no DBConnection instance.
Constructor	Same name as class, no return type. Stores incoming values, builds components and registers listeners.
Array and index	Positions start at 0. Parallel arrays keep the displayed row aligned with its database ID.
try / catch / throws	try performs fallible work; catch handles errors; throws lets the caller handle an error.
dispose() / System.exit(0)	dispose closes one window; System.exit terminates the entire Java process.
== / equals()	Button objects are compared by identity with ==. String values use equals(), including case-sensitive passwords.

The repeated screen-construction pattern

Most constructors follow this order: choose GridLayout; create Label/TextField/Button/Panel objects; set colors/fonts; attach listeners; add components; add a WindowListener; set title, size and position; make the screen visible; optionally load database data. Labels display text, TextFields accept one line, TextArea displays multiple lines, Choice is a dropdown, and java.awt.List is a selectable list.

GridLayout divides a container into equal cells. Panels allow one region to have its own layout. setLocationRelativeTo(null) centers a window. setEditable(false) prevents changing a details TextArea. setEchoChar('*') hides password characters visually; it does not encrypt the password.

How a button click becomes application behavior

```
saveBtn.addActionListener(this);

public void actionPerformed(ActionEvent e) {
    Button btn = (Button) e.getSource();
    if (btn == saveBtn) { /* validate and save */ }
}
```

The source is the component that generated the event. The cast treats it as a Button. An if/else chain routes the event. WindowListener requires seven methods; most are empty because only closing needs custom behavior. MouseListener similarly requires five methods.

The recurring JDBC pattern

```
Connection con = DBConnection.getConnection();
PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM customers WHERE email = ?");
ps.setString(1, user);
ResultSet rs = ps.executeQuery();
if (rs.next()) { /* read columns */ }
con.close();
```

The question mark is a value placeholder. JDBC parameter numbers begin at 1, unlike Java array indexes. executeQuery() returns a ResultSet for SELECT. rs.next() moves to a row and reports whether one exists. executeUpdate() is used for INSERT/UPDATE/DELETE and returns affected-row count. Parameter binding separates values from SQL syntax, helping prevent SQL injection in these queries.

The current code usually closes a connection on its successful path. A production improvement is try-with-resources so connections, statements and results are reliably closed even after exceptions.

05 Startup, database and popups

Source: src/AirlineMain.java; src/DBConnection.java; src/MessageBox.java

These classes create the starting point, centralize database access, and provide reusable modal messages.

AirlineMain() and actionPerformed()

The constructor creates the welcome screen with Admin Login, Customer Login and Exit. It then opens and closes a database connection as a startup check. Failure shows a message explaining that MySQL or its port may need attention. It does not stop the whole application automatically.

The event handler opens LoginFrame with the role string "Admin" or "Customer". Exit and the welcome window close action call System.exit(0), ending the process including its HTTP server.

main() and findButton()

main() installs a global AWT keyboard listener, starts PaymentServer, and constructs AirlineMain. For a Ctrl+A to Ctrl+Z key press, the listener walks up the component parents to the containing Window. findButton() recursively searches nested containers for an enabled button whose trimmed label starts with that letter. It consumes the key and dispatches an ActionEvent.

This is first-match behavior. If several buttons begin with the same letter, it cannot distinguish all of them. The listener is global to this application, not a system-wide keyboard shortcut.

DBConnection.getConnection()

Class.forName loads com.mysql.cj.jdbc.Driver. DriverManager.getConnection opens jdbc:mysql://localhost:3307/airline_db with the configured root user and empty password. localhost means the machine running Java; 3307 is the database port. Change this source setting if your local MySQL uses another port. The method returns a Connection and propagates failures with throws Exception.

MessageBox constructors and actions

MessageBox extends Dialog. Its two-argument constructor delegates to the three-argument constructor with askYesNo=false. super(f, "Message", true) creates a modal dialog owned by the supplied Frame. It splits newline-separated messages into Labels, displays either OK or Yes/No, packs to fit, and centers on the parent.

The yes field starts false. Clicking Yes changes it to true; every button then disposes the dialog. Closing the dialog leaves false. Because it is modal, the caller can inspect new MessageBox(..., true).yes after the user answers. This is used before cancellation and deletion.

Actual source: src/AirlineMain.java, lines 113-129

```
113      static Button findButton(Container box, int key)
114      {
115          Component[] list = box.getComponents();
116          for (int i = 0; i < list.length; i++)
117          {
118              Button b = null;
119              if (list[i] instanceof Button)
120                  b = (Button) list[i];
121              else if (list[i] instanceof Container)
122                  b = findButton((Container) list[i], key);
123
124              if (b != null && b.isEnabled() && b.getLabel().trim().toUpperCase().charAt(0)
    == key)
125                  return b;
126          }
127          return null;
128      }
129
```

06 Login, registration and dashboards

Source: src/LoginFrame.java; src/RegisterFrame.java; src/AdminDashboard.java; src/CustomerDashboard.java

The same login screen serves two roles. The signed-in customer ID is passed into later screens so their queries can select the correct customer records.

LoginFrame(String t)

The role controls the heading, whether the identifier is username or email, the number of layout rows, and whether Register is shown. Password input is masked. actionPerformed() trims the identifier, handles Back/Register first, rejects empty credentials, then queries admins by username or customers by email.

The query retrieves a user record; Java compares the stored password with pass using String.equals(). If rs.next() is false, the || short-circuit avoids reading a nonexistent row. On success, AdminDashboard receives the username, whereas CustomerDashboard receives customer_id and name. The login window disposes itself.

RegisterFrame() and actionPerformed()

The screen contains full name, email, phone, password and confirmation. The handler checks required fields, matches a basic email regex, requires exactly 10 digits for phone, requires at least four password characters, and checks confirmation equality. It then binds four values into INSERT INTO customers and shows success.

The email regex requires a local part, @, a domain and a final alphabetic suffix of at least two characters. This is format checking, not proof that the mailbox exists. The UNIQUE email constraint provides the database-level duplicate check. The catch message covers either an already-used email or another database error.

AdminDashboard

The constructor displays the admin name and three buttons. Manage Flights opens ManageFlightsFrame; View Bookings opens ViewBookingsFrame. Logout opens a new admin login and disposes the dashboard. There are no SQL queries in this dashboard.

CustomerDashboard

The constructor stores customerId and displays the name. View Flights opens AvailableFlightsFrame(customerId); My Bookings opens MyBookingsFrame(customerId). Logout opens customer login and closes the dashboard. The ID carries identity between windows; this is not an HTTP session or a security token.

Security and lifecycle facts to state accurately

Passwords are stored in plain text and compared directly. Masking a TextField is only a display feature. A real system should hash passwords and apply authorization in a service layer. Logging out disposes the dashboard but does not centrally close every child screen already opened from it.

Actual source: src/LoginFrame.java, lines 91-109

```
 91                  String sql = "SELECT * FROM customers WHERE email = ?";
 92                  if (type.equals("Admin"))
 93                      sql = "SELECT * FROM admins WHERE username = ?";
 94                  PreparedStatement ps = con.prepareStatement(sql);
 95                  ps.setString(1, user);
 96                  ResultSet rs = ps.executeQuery();
 97
 98                  // password is checked in java so that it is case sensitive
 99                  if (!rs.next() || !rs.getString("password").equals(pass))
100                      new MessageBox(this, "Wrong login details");
101                  else
102                  {
103                      if (type.equals("Admin"))
104                          new AdminDashboard(user);
105                      else
106                          new CustomerDashboard(rs.getInt("customer_id"),
    rs.getString("name"));
107                      dispose();
108                  }
109                  con.close();
```

07 Managing and saving flights

Source: src/ManageFlightsFrame.java; src/FlightFormFrame.java

The administrator can list, find, create, update and delete flights. Saving a flight can also create or recreate its seat records.

ManageFlightsFrame: constructor and loadFlights(search)

The constructor builds a search area, action buttons and a monospaced list, then calls loadFlights(""). The query returns all rows for an empty search, or an exact flight_no/flight_id match. It orders by departure time and displays at most 500 flights.

flightIds[] maps each list position to its real database ID. String.format pads values into aligned columns; substring(0,16) shortens the displayed date/time. The displayed row is not itself used as the database key.

ManageFlightsFrame.actionPerformed()

Search normalizes input to uppercase; List All clears it. Add opens FlightFormFrame(this,0), where zero means a new flight. Update uses flightIds[selectedIndex]. Update/Delete require a selected row. Delete first asks confirmation, then executes DELETE only if NOT EXISTS a CONFIRMED booking for that flight. Cascades remove related child records, and the list reloads.

FlightFormFrame constructor and input validation

The form stores its parent list window and flightId. For an existing ID it loads the old values into text fields. On Save it parses dates using LocalDateTime.parse after replacing the separating space with T. It parses price as double and seats as int; parse failures show a format message.

It requires nonempty flight number, airline, source and destination; rejects equal source/destination ignoring case; requires arrival after departure; requires positive price and 1-180 seats; and checks future departure for a new flight only. Existing flights are not subject to that final future-date check.

saveFlight(price,seats): update path

The method disables auto-commit. For an existing flight, SELECT ... FOR UPDATE reads total/available seats and counts confirmed bookings while locking the flight. If total capacity is unchanged, it keeps the old available count and skips seat generation. If capacity changes and confirmed bookings exist, it throws an exception. Otherwise it deletes the old seats and prepares to rebuild them.

saveFlight(price,seats): SQL and generated IDs

It selects INSERT for a new flight and UPDATE for an existing one, binds all common fields, and adds the flight ID parameter only for UPDATE. Statement.RETURN_GENERATED_KEYS requests the new AUTO_INCREMENT ID. After execution, getGeneratedKeys() retrieves that ID for the seat inserts.

Seat generation, commit and failure

For i from 0 to seats-1, the label is (i / 6 + 1) followed by ABCDEF.charAt(i % 6). Integer division gives the row; remainder gives the seat letter. For i=0 the result is 1A; i=5 gives 1F; i=6 gives 2A.

The flight and seat changes commit together. The parent list reloads and the form closes. Errors trigger rollback and a message, so partial seat creation does not persist. If the connection was never established, the rollback attempt itself can fail and is caught by the nested catch.

Actual source: src/FlightFormFrame.java, lines 219-227

```
219             // making seats like 1A, 1B ... 1F, 2A ... (6 seats in a row)
220             PreparedStatement ps3 = con.prepareStatement("INSERT INTO seats (flight_id,
    seat_no) VALUES (?, ?)");
221             for (int i = 0; makeSeats && i < seats; i++)
222             {
223                 ps3.setInt(1, id);
224                 ps3.setString(2, (i / 6 + 1) + "" + "ABCDEF".charAt(i % 6));
225                 ps3.executeUpdate();
226             }
227
```

08 Flight browsing and seat selection

Source: src/AvailableFlightsFrame.java; src/SeatSelectionFrame.java

These screens turn a selected flight into a collection of selected seat IDs. No booking is committed at this stage.

AvailableFlightsFrame constructor

It stores the customer ID and builds From/To Choice controls, a list and navigation buttons. A UNION query combines source and destination city names, removes duplicates, and orders them. Both dropdowns begin with All. Then loadFlights("All","All") runs.

loadFlights(from,to) and actionPerformed()

Each filter is optional through (? = 'All' OR source/destination = ?). The query also computes departure_time < NOW() AS departed. The method labels each row Departed, Full or Available and stores its ID and status in parallel arrays, up to 500 rows.

Despite the screen title and README wording, departed flights are displayed; they cannot be selected for booking through this handler. Filter uses current dropdowns. List All resets both. Select Flight requires a row and Available status before opening SeatSelectionFrame.

SeatSelectionFrame constructor and loadSeats()

The constructor keeps references to the flight list, customer and flight; creates a SeatMap; shows flight details, legend, selected-seat summary and buttons. loadSeats() reads the flight price and total capacity, allocates arrays, sets selCount to zero, and reads seats ordered by seat_id. Each map position matches a database seat ID in seatIds[].

After reading, repaint() asks AWT to draw the map and updateSelected() refreshes the text. Reloading clears the current selection, which is useful when another customer has taken a seat.

updateSelected() and Continue

updateSelected() walks selected map indexes and builds a seat-name string. If any are selected, total fare is price times selCount. On Continue, the handler rejects zero seats, creates arrays sized exactly to the selection, copies real database IDs and seat numbers, and passes them to PassengerDetailsFrame.

Example: map.selected[0]=6 means the seventh map entry. seatIds[6] is the database key and map.seatNos[6] may be 2A. These three values have different meanings.

Actual source: src/SeatSelectionFrame.java, lines 139-147

```
139             int[] ids = new int[map.selCount];
140             String[] nos = new String[map.selCount];
141             for (int j = 0; j < map.selCount; j++)
142             {
143                 ids[j] = seatIds[map.selected[j]];
144                 nos[j] = map.seatNos[map.selected[j]];
145             }
146             new PassengerDetailsFrame(this, flightsFrame, customerId, flightId, ids, nos,
    price);
147         }
```

09 How the custom seat map works

Source: src/SeatMap.java

SeatMap extends Canvas and implements MouseListener. It draws the plane and seat rectangles itself instead of creating a Button for each seat.

State and constructor

seatNos[] stores labels; status[] stores AVAILABLE/BOOKED; selected[] has capacity six and stores map indexes; selCount records how many positions are active. seat, gap, aisle, startX and top define the drawing geometry. scale, offX and offY describe the final transformation. The constructor stores the parent and registers mouse events.

seatX(i), seatY(i), findSelected(i)

seatX uses i/6 to move along the aircraft horizontally for each row. seatY uses i%6 to position letters A-F vertically, with extra aisle spacing for D-F. findSelected scans only the first selCount entries and returns a position or -1. It is used both for coloring and toggling selections.

mousePressed(e): hit testing and toggling

Mouse coordinates are screen pixels. The method subtracts offsets and divides by scale to convert them back to the original drawing coordinate system. It then checks whether the point lies inside a seat rectangle and whether that seat is AVAILABLE.

If already selected, it replaces that array entry with the last selected entry and reduces selCount. This is a compact removal operation, so selection order may change. If six seats are already selected, a popup blocks a seventh. Otherwise it appends the index. repaint() redraws and frame.updateSelected() refreshes the total.

paint(g): geometry, scaling and buffering

It computes rows as (seatNos.length+5)/6, which rounds up for an incomplete last row. It computes plane bounds, creates an off-screen Image and obtains Graphics2D. Antialiasing smooths edges. It chooses the smaller width/height fit factor, capped at 1, then translates and scales the graphics.

Ovals, rectangles, arcs and lines form the aircraft. The method draws cockpit windows, FRONT text, seat letters and row numbers. Each seat uses a rounded rectangle: available white, selected green, booked grey with X. It copies the finished buffer to the screen with drawImage and disposes the temporary graphics context.

update(g) and unused mouse callbacks

update(g) directly calls paint(g), avoiding the default background-clearing step that can contribute to flicker. Combined with off-screen drawing, this makes repainting smoother. The four unused mouse methods are empty because MouseListener requires them.

Actual source: src/SeatMap.java, lines 28-40

```
28          int x = (int) ((e.getX() - offX) / scale);
29          int y = (int) ((e.getY() - offY) / scale);
30
31          for (int i = 0; i < seatNos.length; i++)
32          {
33              boolean inside = x >= seatX(i) && x <= seatX(i) + seat && y >= seatY(i) && y <=
    seatY(i) + seat;
34              if (!inside || !status[i].equals("AVAILABLE"))
35                  continue;
36
37              int pos = findSelected(i);
38              if (pos >= 0)
39              {
40                  selected[pos] = selected[selCount - 1]; // last selected seat comes in its
   place
```

Explain it to sir: "The map is drawn in a fixed logical coordinate system, then scaled to fit. I apply the inverse transformation to mouse clicks so hit detection still matches the seats."

10 Passenger details and booking

Source: src/PassengerDetailsFrame.java

This is the most important business-logic class. It collects passenger data and performs the multi-seat booking transaction after demo payment succeeds.

Constructor: one row per selected seat

It stores parent windows, customer/flight IDs, selected seat arrays and price. count equals ids.length. names[], age[] and genders[] hold passenger data. The GridLayout has count+3 rows and four columns: title, headings, passenger rows and buttons. Each passenger row adds a seat Label, name TextField, age TextField and gender Choice.

actionPerformed(): reading dynamic components

The first eight components are the four title cells and four heading cells. Passenger i starts at 8+i*4. Name is +1, age is +2, and gender is +3. The handler casts each component to its expected type, parses age, converts parsing failures to zero, then requires a nonempty name and age 1-120.

It stores the values in arrays and builds the selected-seat text. Finally it opens PaymentFrame with customer ID, flight ID, selected labels, passenger count and price*count. No INSERT into bookings occurs here.

confirmBooking(): start and flight lock

PaymentFrame calls this method after payment success processing. It opens a connection, disables auto-commit and selects the flight price only where departure_time > NOW(), with FOR UPDATE. No row means the flight is missing or no longer eligible; the displayed error says it has departed. The fresh ticket price is read from the database.

Claim seats and reduce availability

For every selected seat ID, UPDATE seats SET status='BOOKED' WHERE seat_id=? AND status='AVAILABLE' must affect exactly one row. Otherwise an exception aborts the entire booking. Then flights.available_seats is reduced by count. The selected state on the Canvas is only a snapshot; this guarded UPDATE is the decisive check.

Generate a PNR and insert passenger rows

TicketService.randomCode(6) generates a candidate. A SELECT checks whether it already appears in bookings; a loop retries if it does. One prepared INSERT is reused for every passenger, binding the same PNR/customer/flight, that passenger's seat and personal details, CONFIRMED status and the freshly read ticketPrice.

The uniqueness check is application-level, not a database uniqueness guarantee across concurrent bookings on different flights. PNR is intentionally shared by passenger rows, so a separate reservation table would be a better place for a unique reservation key.

Commit, rollback and bookingDone()

Only after every seat and passenger succeeds does the method commit, close and return the PNR. Any error rolls back, closes and rethrows, allowing PaymentFrame to report failure. bookingDone() reloads the flight list with All filters and disposes the seat and passenger screens. It is called when closing a completed payment window.

Actual source: src/PassengerDetailsFrame.java, lines 165-174

```
165             // 2. mark every seat BOOKED only if it is still AVAILABLE
166             PreparedStatement ps2 = con.prepareStatement("UPDATE seats SET status =
    'BOOKED' WHERE seat_id = ? AND status = 'AVAILABLE'");
167             for (int i = 0; i < count; i++)
168             {
169                 ps2.setInt(1, seatIds[i]);
170                 if (ps2.executeUpdate() != 1)
171                     throw new Exception("Sorry, seat " + seatNos[i] + " was just booked by
    someone else");
172             }
173
174             // 3. decrease available seats by number of passengers
```

11 Transactions: the key viva topic

A transaction makes several SQL changes behave as one logical operation. con.setAutoCommit(false) begins explicit transaction control. con.commit() saves the changes together; con.rollback() undoes the uncommitted changes.

Step in a two-seat booking	Database effect
Before booking	1A AVAILABLE, 1B AVAILABLE, available_seats=12, no passenger booking rows.
Lock flight	FOR UPDATE serializes competing work that locks that same flight row.
Claim 1A and 1B	Each conditional UPDATE must change exactly one available seat to BOOKED.
Update count	available_seats becomes 10 inside the transaction.
Insert passengers	Two CONFIRMED rows are inserted with one PNR and separate booking_id values.

Step in a two-seat booking	Database effect
Commit	Both seats, the count and both passenger rows become durable together.
If seat 1B cannot be claimed	Rollback also undoes the earlier 1A change. This attempt creates no partial group booking.

Two customers choose the same seat

Both may see 1A as white because selecting does not lock it. Customer A reaches confirmation first and locks the flight, claims 1A and commits. Customer B then gets the flight lock. B's conditional seat UPDATE affects zero rows because 1A is now BOOKED. B's transaction rolls back and the booking fails.

ACID in words you can say

Atomicity: all passenger/seat updates succeed or none persist. **Consistency:** application rules and constraints aim to preserve valid seat and booking relationships. **Isolation:** locks control competing transactions. **Durability:** after commit, the database retains the changes according to its storage configuration.

Where the transaction boundary ends

The payment status change, reservation transaction and later payment-PNR update use separate connections/transactions. Therefore the entire payment-plus-booking process is not atomic. A booking could commit and a later payment update could fail. PDF and email are also outside the reservation transaction; their failure does not cancel a committed booking.

Good presentation wording: "The multi-seat reservation is transactional. The demo payment record and ticket delivery are separate steps, so there are integration failure cases I would improve in a production design."

12 The Java QR payment window

Source: src/PaymentFrame.java

PaymentFrame is the bridge between entered passenger data and a payment attempt. It displays the fare, makes a QR code and receives callbacks from PaymentServer.

Constructor and QR Canvas

It stores customer/flight/passenger information and creates a two-column layout. The anonymous Canvas overrides paint(). For each true cell in the ZXing BitMatrix it draws a black square over a white QR background. box is an integer pixel size chosen to fit the matrix within the Canvas. The constructor ends by calling newPayment().

newPayment()

It makes a PAY-prefixed six-character random reference and inserts a PENDING row with seat labels, passenger count and amount. It builds the URL using PaymentServer.getLaptopIp() and port 5173, encodes it with QRCodeWriter and requests a repaint.

PaymentServer.current=this lets the HTTP handler find this live window. The waiting message is shown and Retry is disabled. This code uses callbacks, not periodic Java polling. Although the encoder is called with width/height 1, ZXing returns a matrix large enough for a valid symbol; the Canvas scales its modules.

paymentSuccess()

First it calls detailsFrame.confirmBooking(). If that throws, it marks the demo payment FAILED, shows an error, reloads seats and rethrows. If booking succeeds, it separately writes the returned PNR into the payment record, sets done=true, updates the label and returns the PNR to the server.

If the PNR update fails after confirmBooking committed, the booking may exist even though the complete success flow did not finish. Also, the booking-failure catch does not enable Retry the way paymentFailed() does.

makeTicket()

After the server replies to the phone, this method calls makePdf(pnr), then sendEmail(pnr,path), updates the status text and renames Cancel to Close. Finally it clears PaymentServer.current. The catch labels any error in this block as Email failed, even if PDF creation was the failing operation.

paymentFailed(), actionPerformed(), closeWindow()

paymentFailed() turns the status red and enables Retry. Retry invokes newPayment(), producing a new reference and QR; the old failed row remains. Other button actions close the window.

If done=true, closeWindow calls bookingDone() on the passenger screen. Otherwise it changes only a still-PENDING payment to FAILED and clears current. Then it disposes the payment screen. Cancelling this pending window does not run the ticket-cancellation workflow and does not reserve seats.

Actual source: src/PaymentFrame.java, lines 123-130

```
123             String url = "http://" + PaymentServer.getLaptopIp() + ":5173/?ref=" + ref;
124             qr = new QRCodeWriter().encode(url, BarcodeFormat.QR_CODE, 1, 1);
125             qrCanvas.repaint();
126             PaymentServer.current = this; // so the server can tell this window when
    payment is done
127             l3.setText("Payment Ref: " + ref);
128             statusLabel.setForeground(new Color(200, 120, 0));
129             statusLabel.setText("Waiting for payment on phone...");
130             retryBtn.setEnabled(false);
```

13 The Java HTTP payment server

Source: src/PaymentServer.java

The server listens on port 8085 and exposes JSON responses to the phone website. It is a local demonstration backend, not a real payment processor.

start()

HttpServer.create binds port 8085. createContext("/api/payment",handler) routes matching paths to an anonymous HttpHandler, whose handle() delegates to handleRequest(). server.start() begins accepting requests. Startup errors are logged, but the desktop welcome screen can still open.

handleRequest(ex)

It reads the URI path and raw query. If the query begins with ref=, it takes the remaining text, uppercases it and removes characters other than A-Z and 0-9. This is a simple parser tailored to one parameter, not a general URL-query parser. It compares the reference with current.ref to obtain the matching PaymentFrame.

The path suffix selects details, success or fail handling. For fail, a conditional database UPDATE changes only PENDING to FAILED; if one row changed, the matching window gets paymentFailed(). It writes UTF-8 JSON, Content-Type application/json, Access-Control-Allow-Origin: *, and HTTP status 200. Logical success/failure is carried in the JSON ok field.

After writing the response, a successful path with a completed frame calls makeTicket(). This happens after the response but still on the handler execution path, not on a newly created background worker.

getDetails(ref)

A parameterized JOIN reads payments, flights and customers for the reference. It manually builds JSON containing status, amount, seat labels, count, flight/route/departure, customer and PNR. Missing rows produce ok:false. This query reads payment details, not the whole reservation ticket.

paymentDone(ref,pf)

If no matching open frame exists, the method marks a pending reference FAILED and returns an error. Otherwise it changes status from PENDING to SUCCESS only if still pending. An affected-row result other than one blocks repeated success handling. It then calls pf.paymentSuccess() and returns its PNR; caught exceptions return an error message.

The status guard prevents ordinary double-clicks from creating duplicate bookings for the same attempt. It does not make the payment update and booking insert one database transaction.

update(), getLaptopIp(), message(), esc()

update() opens a connection, binds one reference, executes the supplied UPDATE, closes and returns row count; on exception it logs and returns zero. getLaptopIp() uses a connected UDP socket to infer the local address used for a route; connecting it does not itself send payment data. It falls back to localhost, which a phone would interpret as the phone itself.

message() constructs the common ok/message JSON shape. esc() replaces backslashes and quotation marks and converts null to empty text. It is not a full JSON serializer: other control characters are not handled.

Known design limits

Only one static current payment window is tracked. Opening another can replace the pointer. The handler routes by path but does not enforce GET versus POST, even though React uses those intended methods. There is no login token or verified bank callback. AWT components are also updated from HTTP callbacks rather than marshalled onto the AWT event-dispatch thread. These are reasons to present this as a student demo integration.

Intended request	What it does
GET /api/payment/details?ref=...	Returns JSON payment and associated flight/customer details.
POST /api/payment/success?ref=...	Attempts pending-to-success transition, then reservation confirmation.
POST /api/payment/fail?ref=...	Marks a pending attempt failed and enables desktop Retry when the matching frame exists.

14 The React payment website

Source: payment-website/src/App.jsx; src/main.jsx; src/App.css; vite.config.js

The website displays one payment reference. It calls the Java server; it never accesses MySQL directly.

index.html and main.jsx: entering React

index.html defines metadata, a responsive viewport and an empty div with id="root". Its module script loads /src/main.jsx. main.jsx imports React, ReactDOM, App and the stylesheet. createRoot(document.getElementById("root")).render(<App />) mounts the application into that div.

App(): URL and state

API is http:// plus window.location.hostname plus :8085/api/payment. Using the page hostname keeps the phone pointed at the laptop rather than its own localhost. URLSearchParams extracts ref from the QR URL.

Four useState hooks store payment data, an error, an informational note and a busy flag. Their setter functions update state and cause React to render the changed view. useEffect with [] invokes loadDetails after the initial mount; there is no recurring polling timer.

loadDetails()

Without a reference it sets a user-facing error and returns. Otherwise fetch calls /details?ref=..., res.json() parses the JSON, and the next then stores either data or its error message. catch handles connectivity/parsing failures with a same-Wi-Fi hint. This is asynchronous: the browser remains responsive while waiting.

send(action)

It sets busy=true, sends a POST to /success or /fail, parses the response, displays data.message, clears busy and reloads details. Network failure sets a note and clears busy. The comment mentions retry, but the rendered page only sends success/fail. Actual retry happens on the laptop with a new reference.

JSX rendering and conditional screens

The header always renders. error && renders the error card when there is an error. !error && !payment shows a loading message. payment && renders route, flight, seats, passenger count and customer information when data exists. amount.toFixed(2) displays two decimal places.

When status is PENDING, the two buttons appear and disabled={busy} reduces repeated clicks while waiting. Arrow functions call send("success") or send("fail") only on click. SUCCESS shows the PNR. FAILED tells the user to retry on the laptop. The success text claims email delivery, but the server response is sent before email is attempted, so the browser has not verified delivery.

App.css: styling block by block

The :root variables define navy, light, green and red; * applies border-box sizing. body sets font, background and base color. .page centers a narrow 480px layout. .header/.logo/.sub style the brand. .card and .card.error create content/error panels; .ref styles the reference.

.route uses flex layout to separate cities and plane icon; .city/.small/.plane control typography. .details styles the table, row borders, muted left labels and bold right values. .amount and its span emphasize fare. .buttons uses flex with gaps; button defines shared size/shape; :disabled lowers opacity; .done/.fail set status colors. .retry is defined but unused by the current JSX.

.result and its first bold child style status messages; .success/.failed provide colored backgrounds; .note and footer style secondary text. CSS changes appearance, not booking rules.

package.json, package-lock.json and Vite

package.json declares React/ReactDOM 18.3.1 and development dependencies Vite 5.4.11 and the React plugin 4.3.4. The dev script runs vite --host; build runs vite build. The lockfile records the exact resolved dependency graph.

vite.config.js enables the React plugin and exposes the development server to the network on port 5173. If that port is occupied, a development server may select another port; the Java QR hardcodes 5173, so confirm the displayed port before your demo.

Actual source: payment-website/src/App.jsx, lines 35-49

```
35    function send(action) {
36      setBusy(true)
37      fetch(API + '/' + action + '?ref=' + ref, { method: 'POST' })
38        .then((res) => res.json())
39        .then((data) => {
40          setNote(data.message)
41          setBusy(false)
42          loadDetails()
43        })
44        .catch(() => {
45          setNote('Cannot connect to the airline server.')
46          setBusy(false)
47        })
48    }
49
```

15 Ticket PDF, email and random codes

Source: src/TicketService.java

This utility class supplies shared reservation details to the admin/customer screens, the PDF and the email body.

getDetails(pnr)

A JOIN across bookings, seats, customers and flights retrieves every passenger row for a PNR, ordered by seat_id. It creates the common flight/customer header from the first row only, appends each passenger's name, age/gender, seat, status and price, counts rows, and adds only CONFIRMED fares to the total.

Cancelled passengers remain listed, but their fares are excluded from Total Fare. If no row exists, the method throws Booking not found. It returns a single multiline String, which is reused in several outputs.

makePdf(pnr)

It splits getDetails() into lines, ensures the tickets directory exists, and chooses tickets/ticket_PNR.pdf. PDDocument is the PDF container; PDPage is one page; PDPageContentStream draws on it. The heading uses Helvetica Bold, the subtitle Helvetica, and the details Courier so padded columns line up.

PDF text is drawn with beginText/endText, setFont, newLineAtOffset and showText. setLeading(16) controls vertical line spacing; newLine() advances the cursor. addRect/stroke draw a surrounding box. The content stream closes, the document saves and closes, and the relative path is returned. Regenerating the same PNR writes the same filename.

PDF limitations

The ticket generator creates one fixed page, with no wrapping/pagination logic. Long names or lines can overflow, and standard fonts have limited character support. The printed phrase Boarding Pass is a project label; this is not an airline-issued boarding pass or a real check-in system.

sendEmail(pnr,pdfPath)

It reads mail.username and mail.password from config.properties. Empty values raise an error. It queries the booking customer's email, then configures SMTP authentication, STARTTLS, smtp.gmail.com and port 587. An anonymous Authenticator returns PasswordAuthentication with the configured credentials.

A MimeMessage receives From, To and Subject. One MimeBodyPart contains greeting plus reservation text; another attaches the saved PDF. MimeMultipart combines them, setContent assigns the combined body, and Transport.send attempts delivery. The booking is already committed, so SMTP failure does not release seats.

randomCode(length)

It chooses random characters from a custom alphabet omitting several visually confusing letters. A for loop appends length characters using java.util.Random. Six characters are used for PNRs and the variable part of payment references. This is a convenience identifier generator, not a cryptographically secure token generator.

Actual source: src/TicketService.java, lines 139-148

```
139          MimeBodyPart textPart = new MimeBodyPart();
140          textPart.setText("Dear " + name + ",\n\nThank you for booking with us. Your ticket
    is attached.\n\n" + getDetails(pnr) + "\n\nHappy journey!\nSkyWay Airlines");
141          MimeBodyPart filePart = new MimeBodyPart();
142          filePart.attachFile(new File(pdfPath));
143          Multipart mp = new MimeMultipart();
144          mp.addBodyPart(textPart);
145          mp.addBodyPart(filePart);
146          msg.setContent(mp);
147          Transport.send(msg);
148      }
```

16 Viewing bookings and cancellation

Source: src/MyBookingsFrame.java; src/ViewBookingsFrame.java

Both screens display passenger-level rows but show group-level ticket details for the selected row's PNR. Only the customer screen includes cancellation and ticket delivery actions.

MyBookingsFrame constructor and loadBookings(pnr)

The screen stores customerId and parallel arrays for PNRs, booking IDs and passenger names, capped at 500. Its query joins bookings with seats and flights and always filters b.customer_id=?. An optional PNR filter sits inside parentheses so it cannot bypass the customer condition. Rows sort by booking date, PNR and seat ID.

The list shows one passenger per row. The details area initially reports total rows. itemStateChanged() gets the selected row's PNR and calls TicketService.getDetails(), which shows all passengers under that PNR.

actionPerformed(): navigation, ticket and confirmation

Back closes the window. Search uppercases the supplied PNR; List All clears it. Other actions require a selected row. Get Ticket calls makePdf and sendEmail again, then displays success/error text. The PDF can still have been saved if only email failed. The catch text says Email failed for any exception in that block.

Cancel asks a modal Yes/No question using the selected passenger name. A Yes invokes cancelTicket(i). It cancels by booking_id, not by shared PNR, so the other passengers remain confirmed.

cancelTicket(i): guarded transaction

The method disables auto-commit and selects the booking joined with its flight, restricted by both booking_id and customer_id, FOR UPDATE. It rejects missing or non-CONFIRMED bookings and departures earlier than NOW().

It updates the booking status to CANCELLED, sets its seat AVAILABLE, and increments the flight's available count only while below total capacity. It commits, closes and reloads the list. Any exception rolls back and displays a message. A second cancellation finds the status already cancelled and cannot increment the count twice.

What cancellation does not do

It does not delete the passenger row or cancel the whole PNR. It does not call a refund gateway or update the payment amount/status. After cancellation, regenerated reservation details exclude that passenger's fare from the active total but still show their CANCELLED record.

ViewBookingsFrame constructor and loadBookings(pnr)

The administrator's screen uses the same general layout and JOIN pattern but has no customer restriction and supports up to 1,000 rows. Search optionally filters by exact PNR. actionPerformed handles Search/List All/Back. itemStateChanged calls TicketService.getDetails() for the selected PNR. There are no admin cancellation buttons in this class.

Actual source: src/MyBookingsFrame.java, lines 172-183

```
172             // 1. lock the booking row and check it
173             PreparedStatement ps1 = con.prepareStatement("SELECT b.*, f.departure_time <
    NOW() AS departed FROM bookings b "
174                     + "JOIN flights f ON b.flight_id = f.flight_id WHERE b.booking_id = ?
    AND b.customer_id = ? FOR UPDATE");
175             ps1.setInt(1, bookingIds[i]);
176             ps1.setInt(2, customerId);
177             ResultSet rs = ps1.executeQuery();
178             if (!rs.next() || !rs.getString("status").equals("CONFIRMED"))
179                 throw new Exception("This ticket is already cancelled");
180             if (rs.getInt("departed") == 1)
181                 throw new Exception("Flight has already departed, cannot cancel");
182
183             // 2. mark booking cancelled, free the seat and increase available seats
```

17 Setup, dependencies and demo checks

Run commands from the project root unless a command explicitly changes directory. The Java source was successfully compiled against the supplied libraries during preparation of this guide. Compilation verifies syntax/types and library availability; it does not prove database connectivity, phone networking or email delivery.

Java compilation passed. The React production build was attempted but blocked by macOS rejecting the installed Rollup native module's code signature (ERR_DLOPEN_FAILED). This is a local dependency-loading failure, not an observed JSX compile error. Before the presentation, reinstall the web dependencies from a trusted npm installation if needed and verify npm run dev. No full live payment or email test was performed.

Local startup order

1. Start the local MySQL service and verify the connection settings in DBConnection.java (currently port 3307, database airline_db). For a fresh database only, import database/airline_db.sql. The script drops existing tables. If an existing database only lacks payments, use payments_table.sql instead.

2. In a terminal, enter payment-website and run npm install if dependencies are missing, then npm run dev. Confirm it is reachable on port 5173. The web page and Java backend must both be running for the QR demo.

3. In another terminal at the project root, run ./run.sh on Mac/Linux or run.bat on Windows. Keep the terminal visible for startup/database messages. The Java app starts its payment server automatically.

4. Put the phone and laptop on the same reachable network. Allow the local Java/Node connections when the operating system requires it. The phone must be able to reach the laptop at both ports 5173 and 8085.

5. Configure your own mail credentials locally only if demonstrating email. The source expects config.properties in the working directory. Never put a real password on a presentation slide. A failed email can be retried with Get Ticket after configuration is corrected.

```
Mac/Linux run.sh:
javac -d classes -cp "lib/*" src/*.java
java -cp "classes:lib/*" AirlineMain

Windows run.bat uses:
java -cp "classes;lib/*" AirlineMain
```

-d chooses the compiled output folder. -cp supplies the classpath. The wildcard includes JAR dependencies. The runtime classpath contains both compiled classes and libraries. The separator is a colon on Mac/Linux and a semicolon on Windows. The shell script uses && so Java runs only when compilation succeeds; the batch file checks errorlevel.

Supplied JAR	Purpose
mysql-connector-j-8.4.0.jar	MySQL JDBC driver.
pdfbox-app-3.0.3.jar	PDFBox PDF generation and bundled dependencies.
core-3.5.3.jar	ZXing QR encoding.
jakarta.mail-api-2.1.3.jar + angus-mail-2.0.3.jar	Mail API plus implementation.
jakarta.activation-api-2.1.3.jar + angus-activation-2.0.2.jar	Attachment/content-type support API plus implementation.

Other project files

.vscode/settings.json identifies src, classes and lib/**/*.jar for Java tooling. config.properties.example is a safe template for real local mail configuration. .gitignore excludes generated/local material from version control. classes and payment-website/dist contain generated output, while tickets stores generated ticket PDFs. The existing SkyWay_Project_Report.pdf is a prior document, not executable code.

Fresh-database sample logins

Role	Seeded credentials
Admin	Username: admin | Password: admin123
Customer	Email: rahul@gmail.com | Password: 1234

These are sample values from the SQL, not a claim about the current live database. The seeded flights are in December 2026, which is after the intended 21 October 2026 presentation date. Confirm the records and computer/database clocks before demonstrating.

18 Present it to sir: a clear script

Opening: about 45 seconds

"My project is SkyWay Airline Reservation Management System. It has two roles: admin and customer. The desktop interface is built in Java AWT, and MySQL stores users, flights, seats and bookings through JDBC. I also integrated a React demo payment page, QR-code generation, PDF tickets and email delivery."

Architecture: about one minute

"Each screen is a Java class. DBConnection creates database connections. The customer ID and selected flight/seat IDs are passed between screen constructors. PaymentServer is a small Java HTTP server that lets the phone page communicate with the desktop application. TicketService generates the reservation text, PDF and email."

Demonstration: about three minutes

Show admin login, list flights and explain Add/Update without changing an important existing flight. Then show customer login, filter a route, choose two seats, and enter two passengers. Point out that green means selected locally, not booked yet. Open payment and scan the QR. Say explicitly that Payment Done simulates payment; no money moves. Show the confirmed PNR and the generated PDF. In My Bookings, show both passengers under one PNR and cancel one passenger to demonstrate seat release.

Most important code: about two minutes

"The critical method is PassengerDetailsFrame.confirmBooking(). It starts a transaction, locks the flight using FOR UPDATE, changes each seat only if it is AVAILABLE, updates the available count, generates a PNR and inserts one row per passenger. It commits only when all steps succeed. If a seat was taken meanwhile, it rolls back all changes. Cancellation also uses a transaction and checks the existing status before releasing the seat."

Close: about 30 seconds

"The project demonstrates event handling, inheritance and interfaces, arrays, custom graphics, JDBC, SQL joins and transactions, HTTP/JSON integration, PDF generation and email. Future improvements include password hashing, service-layer separation, robust payment state management and support for multiple simultaneous payment sessions."

If something fails during the demo

Symptom	What to check / how to explain
Database connection error	MySQL running, database name, port 3307 versus local configuration, driver JAR.
Phone cannot load page	Vite running at 5173; same reachable network; correct laptop IP; local firewall.
Page loads but data fails	Java PaymentServer on 8085; valid reference; Java/DB logs.
Payment says closed	The matching PaymentFrame must still be open; another payment window may have replaced current.
Booking fails after selection	Seat selection is not a hold. Another booking or a departed flight can invalidate it.
Email fails after confirmation	Reservation may already be committed and PDF saved. Check local SMTP configuration and use Get Ticket to retry.

19 Viva questions with model answers

Why Java AWT?

The project uses AWT components for desktop screens and Canvas/Graphics2D for custom drawings. AWT also supplies the event/listener model used throughout.

What is event-driven programming?

The program waits for actions such as a button click or mouse press. Registered listeners receive those events and execute the relevant handler.

Where are inheritance and interfaces used?

Screens extend Frame, MessageBox extends Dialog and SeatMap extends Canvas. Classes implement listener interfaces such as ActionListener, WindowListener and MouseListener.

Where is polymorphism visible?

AWT calls overridden methods such as paint() and interface callbacks such as actionPerformed() on concrete objects. MessageBox also demonstrates constructor overloading.

Why PreparedStatement?

It binds values separately from SQL text, avoids hand-quoting input, and supports safe parameterized queries. Its executeUpdate row count also helps enforce state transitions.

Why not use a plain Statement everywhere?

Statement is used for the fixed city UNION query. Queries with user values use placeholders in PreparedStatement.

What is a PNR?

A reservation reference shared by one group. Each passenger still has a separate booking_id, enabling individual cancellation.

Why store passenger details separately from the customer?

The account holder may book for family or friends. A customer is the purchaser; a passenger is the person assigned to a seat.

How do you prevent double booking?

At confirmation the flight is locked and each seat UPDATE requires AVAILABLE status. If it affects zero rows, the transaction fails and rolls back.

Does selecting a seat reserve it?

No. The green selection is in memory. MySQL changes only after the payment-success callback invokes confirmBooking().

What does FOR UPDATE do?

It requests locks on selected records inside a transaction so conflicting modifications must coordinate with that transaction.

Why rollback?

It prevents partial results, such as one of two seats being booked without the second passenger or without the correct count.

Why can the PNR column not simply be unique?

Several passenger rows intentionally share it. A unique reservation key belongs in a separate reservation/group table with passenger rows referencing it.

What is in the QR code?

The local payment website URL plus a payment reference query parameter. The site uses the reference to fetch details.

Is the payment real?

No. The user clicks success/failure on a demo website. There is no bank/UPI gateway, settlement or refund processing.

How does React talk to Java?

Browser fetch calls return JSON from the Java HttpServer on port 8085. The web page itself is served on 5173.

Why CORS?

Different ports create different browser origins. The Java response permits cross-origin reads with Access-Control-Allow-Origin.

Is email failure the same as booking failure?

No. Booking commits first. PDF/email are later delivery steps; email failure does not undo a confirmed reservation.

What happens if I cancel twice?

The method requires CONFIRMED status while locking the row. After the first commit, another attempt is rejected, so the count is not incremented twice.

What happens to the other passengers when one is cancelled?

Only the selected booking_id is changed. Other passenger rows sharing the PNR remain unchanged.

Why are there both total_seats and available_seats?

Total is capacity. Available is the remaining count maintained alongside individual seat states for quick display. Transactions aim to keep them consistent.

How is 2A generated?

For zero-based i=6, integer division i/6+1 gives row 2, and i%6 gives letter index 0, which is A.

Why use Courier in the ticket?

It is monospaced, so spaces inserted by String.format align text columns.

How are wrong ages handled?

Parsing failure becomes zero, then the validation rejects values outside 1-120.

What would you improve first?

Password hashing; service-layer authorization; consistent payment/booking transaction handling; per-reference sessions; proper threading/resource cleanup; JSON serialization; and robust PDF wrapping.

20 Limitations and accurate claims

Use this section if your examiner asks about weaknesses. These are observations about the current code, not features already implemented.

Current behavior	Practical implication / next improvement
Plain-text account passwords	Masking input is not secure storage. Use salted password hashing and stronger validation.
One PaymentServer.current pointer	Only one matching payment frame is tracked. Use a per-reference session registry or backend-owned workflow.
Payment and reservation saved separately	Failures can leave inconsistent integration state. Design a coordinated state machine with recoverable transitions.
Fare read before payment and again at booking	An admin price change can make the shown payment amount differ from stored ticket_price. Bind and validate a quote.
PNR checked by SELECT only	Rare concurrent collisions are not globally prevented. Use a unique group/reservation record.
No verified payment authentication	Success/failure endpoints simulate events. A real gateway needs authenticated, verified callbacks.
HTTP callbacks modify AWT screens	Use EventQueue.invokeLater for UI updates and background workers for slow work.
Manual resource/JSON handling	Use try-with-resources and a JSON library to handle failures and escaping robustly.
Array-based display limits	Lists cap at 500 or 1,000 records. Use dynamic collections and pagination for larger datasets.
Flight/seat cascades remove history	Preserve historical records if auditability is required.
Fixed single-page ticket layout	Add wrapping, Unicode fonts and pagination for long content.
No automatic reservation expiry/refund flow	Pending attempts and cancellation do not implement a real airline payment lifecycle.

Do not repeat misleading comments as behavior

Some source comments mention "no layout manager", but the actual statements use GridLayout. AvailableFlightsFrame includes departed flights and blocks booking them. PaymentFrame is notified by callbacks rather than polling. The React comment mentions retry, but only the desktop creates a fresh retry. The website success text does not verify email delivery. Always explain the executed statements, not just comments.

Appendix Complete numbered source

The following listings reproduce the handwritten source and relevant setup files reviewed for this guide. Line numbers refer to the original file, not the PDF. Long lines wrap with a continuation marker; the wrapped continuation is still the same original source line. Start with the corresponding explanation chapter, then read the actual method here.

Repeated color, font, component-add and empty listener lines are explained by the common AWT pattern in Chapter 04. External library source, generated files, package-lock entries and private config.properties values are intentionally not duplicated. config.properties.example supplies the safe configuration template.

src/AirlineMain.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // first window of the project
 6 | public class AirlineMain extends Frame implements ActionListener, WindowListener
 7 | {
 8 |
 9 |     Label l1, l2;
10 |     Button adminBtn, customerBtn, exitBtn;
11 |
12 |     public AirlineMain()
13 |     {
14 |         // setting the layout
15 |         setLayout(new GridLayout(5, 1, 10, 10));
16 |
17 |         // creating objects
18 |         l1 = new Label("SKYWAY AIRLINES", Label.CENTER);
19 |         l2 = new Label("Please choose how you want to login", Label.CENTER);
20 |         adminBtn = new Button("Admin Login");
21 |         customerBtn = new Button("Customer Login");
22 |         exitBtn = new Button("Exit");
23 |
24 |         // design
25 |         setBackground(new Color(232, 240, 250));
26 |         setFont(new Font("Arial", Font.PLAIN, 15));
27 |         l1.setBackground(new Color(22, 54, 110));
28 |         l1.setForeground(Color.white);
29 |         l1.setFont(new Font("Arial", Font.BOLD, 26));
30 |         l2.setFont(new Font("Arial", Font.BOLD, 15));
31 |
32 |         // event handling
33 |         adminBtn.addActionListener(this);
34 |         customerBtn.addActionListener(this);
35 |         exitBtn.addActionListener(this);
36 |
37 |         // adding components
38 |         add(l1);
39 |         add(l2);
40 |         add(adminBtn);
41 |         add(customerBtn);
42 |         add(exitBtn);
43 |
44 |         // closing the window
45 |         addWindowListener(this);
46 |
47 |         // creating the frame
48 |         setTitle("SkyWay Airlines");
49 |         setSize(450, 360);
50 |         setLocationRelativeTo(null);
51 |         setVisible(true);
52 |
53 |         // checking the database connection
54 |         try
55 |         {
```

```
56 |             Connection con = DBConnection.getConnection();
57 |             System.out.println("Database connected successfully!");
58 |             con.close();
59 |         }
60 |         catch (Exception e)
61 |         {
62 |             System.out.println(e);
63 |             new MessageBox(this, "Could not connect to MySQL.\nStart MySQL in XAMPP and check the port
   >  in DBConnection.java");
64 |         }
65 |     }
66 |
67 |     // event handling method
68 |     public void actionPerformed(ActionEvent e)
69 |     {
70 |         Button btn = (Button) e.getSource();
71 |
72 |         if (btn == adminBtn)
73 |             new LoginFrame("Admin");
74 |         else if (btn == customerBtn)
75 |             new LoginFrame("Customer");
76 |         else
77 |             System.exit(0);
78 |     }
79 |
80 |     public static void main(String args[])
81 |     {
82 |         // keyboard shortcut: Ctrl + first letter of a button clicks that button (in every window)
83 |         Toolkit.getDefaultToolkit().addAWTEventListener(new AWTEventListener()
84 |         {
85 |             public void eventDispatched(AWTEvent ev)
86 |             {
87 |                 KeyEvent ke = (KeyEvent) ev;
88 |                 int key = ke.getKeyCode();
89 |                 if (ke.getID() != KeyEvent.KEY_PRESSED || !ke.isControlDown() || key < 'A' || key >
   > 'Z')
90 |                     return;
91 |
92 |                 // finding the window in which the key was pressed
93 |                 Component c = ke.getComponent();
94 |                 while (c != null && !(c instanceof Window))
95 |                     c = c.getParent();
96 |
97 |                 Button btn = null;
98 |                 if (c != null)
99 |                     btn = findButton((Container) c, key);
100 |                if (btn != null)
101 |                {
102 |                    ke.consume();
103 |                    btn.dispatchEvent(new ActionEvent(btn, ActionEvent.ACTION_PERFORMED,
    > btn.getLabel()));
104 |                }
105 |            }
106 |        }, AWTEvent.KEY_EVENT_MASK);
107 |
108 |        PaymentServer.start(); // small HTTP server for the demo QR payment website
109 |        new AirlineMain();
110 |    }
111 |
112 |    // searches all panels of the window for a button whose label starts with the letter
113 |    static Button findButton(Container box, int key)
114 |    {
115 |        Component[] list = box.getComponents();
116 |        for (int i = 0; i < list.length; i++)
117 |        {
118 |            Button b = null;
119 |            if (list[i] instanceof Button)
120 |                b = (Button) list[i];
121 |            else if (list[i] instanceof Container)
122 |                b = findButton((Container) list[i], key);
123 |
124 |            if (b != null && b.isEnabled() && b.getLabel().trim().toUpperCase().charAt(0) == key)
125 |                return b;
126 |        }
```

```
127 |         return null;
128 |     }
129 |
130 |     // window events (only closing is used, WindowListener needs all 7 methods)
131 |     public void windowClosing(WindowEvent we)
132 |     {
133 |         System.exit(0);
134 |     }
135 |     public void windowOpened(WindowEvent we) {}
136 |     public void windowClosed(WindowEvent we) {}
137 |     public void windowIconified(WindowEvent we) {}
138 |     public void windowDeiconified(WindowEvent we) {}
139 |     public void windowActivated(WindowEvent we) {}
140 |     public void windowDeactivated(WindowEvent we) {}
141 | }
```

src/DBConnection.java

```
  1 | import java.sql.*;
  2 |
  3 | // every window calls DBConnection.getConnection() to talk to MySQL
  4 | public class DBConnection
  5 | {
  6 |     private static Integer workingPort = null;
  7 |
  8 |     public static Connection getConnection() throws Exception
  9 |     {
 10 |         // Load MySQL JDBC Driver
 11 |         Class.forName("com.mysql.cj.jdbc.Driver");
 12 |
 13 |         if (workingPort != null)
 14 |         {
 15 |             try
 16 |             {
 17 |                 return DriverManager.getConnection("jdbc:mysql://localhost:" + workingPort +
    > "/airline_db", "root", "");
 18 |             }
 19 |             catch (Exception ignored)
 20 |             {
 21 |                 workingPort = null;
 22 |             }
 23 |         }
 24 |
 25 |         // Try standard port 3306 first, then fallback to 3307 (common in custom/XAMPP setups)
 26 |         int[] ports = { 3306, 3307 };
 27 |         Exception lastEx = null;
 28 |         for (int p : ports)
 29 |         {
 30 |             try
 31 |             {
 32 |                 Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:" + p +
    > "/airline_db", "root", "");
 33 |                 workingPort = p;
 34 |                 return conn;
 35 |             }
 36 |             catch (Exception e)
 37 |             {
 38 |                 lastEx = e;
 39 |             }
 40 |         }
 41 |         throw (lastEx != null) ? lastEx : new SQLException("Unable to connect to MySQL on ports 3306
    > or 3307");
 42 |     }
 43 | }
```

src/MessageBox.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 |
 4 | // AWT does not have JOptionPane so this is our own popup box
 5 | public class MessageBox extends Dialog implements ActionListener, WindowListener
 6 | {
 7 |     Button okBtn, yesBtn, noBtn;
 8 |     public boolean yes = false; // true if user clicks Yes
 9 |
10 |     // simple message with OK button
11 |     public MessageBox(Frame f, String msg)
12 |     {
13 |         this(f, msg, false);
14 |     }
15 |
16 |     // if askYesNo is true, it shows Yes and No buttons
17 |     public MessageBox(Frame f, String msg, boolean askYesNo)
18 |     {
19 |         super(f, "Message", true);
20 |
21 |         // setting the layout
22 |         setLayout(new GridLayout(0, 1, 5, 5));
23 |
24 |         // creating objects
25 |         okBtn = new Button("   OK   ");
26 |         yesBtn = new Button("   Yes   ");
27 |         noBtn = new Button("   No   ");
28 |
29 |         // design
30 |         setBackground(new Color(232, 240, 250));
31 |         setFont(new Font("Arial", Font.PLAIN, 15));
32 |
33 |         // event handling
34 |         okBtn.addActionListener(this);
35 |         yesBtn.addActionListener(this);
36 |         noBtn.addActionListener(this);
37 |
38 |         // adding components (one label for every line of the message)
39 |         String[] lines = msg.split("\n");
40 |         for (int i = 0; i < lines.length; i++)
41 |             add(new Label("     " + lines[i] + "     ", Label.CENTER));
42 |         if (askYesNo)
43 |         {
44 |             add(yesBtn);
45 |             add(noBtn);
46 |         }
47 |         else
48 |             add(okBtn);
49 |
50 |         // closing the window
51 |         addWindowListener(this);
52 |
53 |         pack();
54 |         setSize(Math.max(320, getWidth()), getHeight());
55 |         setLocationRelativeTo(f);
56 |         setVisible(true); // program waits here till user clicks a button
57 |     }
58 |
59 |     // event handling method
60 |     public void actionPerformed(ActionEvent e)
61 |     {
62 |         Button btn = (Button) e.getSource();
63 |         if (btn == yesBtn)
64 |             yes = true;
65 |         dispose();
66 |     }
67 |
68 |     // window events (only closing is used, WindowListener needs all 7 methods)
69 |     public void windowClosing(WindowEvent we)
70 |     {
71 |         dispose();
72 |     }
```

```
73 |     public void windowOpened(WindowEvent we) {}
74 |     public void windowClosed(WindowEvent we) {}
75 |     public void windowIconified(WindowEvent we) {}
76 |     public void windowDeiconified(WindowEvent we) {}
77 |     public void windowActivated(WindowEvent we) {}
78 |     public void windowDeactivated(WindowEvent we) {}
79 | }
```

src/LoginFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // one login window for both admin and customer
 6 | public class LoginFrame extends Frame implements ActionListener, WindowListener
 7 | {
 8 |     Label l1, l2, l3, l4;
 9 |     TextField userTf, passTf;
10 |     Button loginBtn, registerBtn, backBtn;
11 |     String type; // "Admin" or "Customer"
12 |
13 |     public LoginFrame(String t)
14 |     {
15 |         type = t;
16 |
17 |         // setting the layout (2 columns, like label and text field)
18 |         if (type.equals("Admin"))
19 |             setLayout(new GridLayout(4, 2, 0, 10));
20 |         else
21 |             setLayout(new GridLayout(5, 2, 0, 10));
22 |
23 |         // creating objects
24 |         l1 = new Label("   " + type.toUpperCase() + " LOGIN");
25 |         l4 = new Label("");
26 |         if (type.equals("Admin"))
27 |             l2 = new Label("    Username");
28 |         else
29 |             l2 = new Label("    Email");
30 |         l3 = new Label("    Password");
31 |         userTf = new TextField(20);
32 |         passTf = new TextField(20);
33 |         loginBtn = new Button("Login");
34 |         registerBtn = new Button("Register (new user)");
35 |         backBtn = new Button("Back");
36 |
37 |         // design
38 |         setBackground(new Color(232, 240, 250));
39 |         setFont(new Font("Arial", Font.PLAIN, 15));
40 |         l1.setBackground(new Color(22, 54, 110));
41 |         l1.setForeground(Color.white);
42 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
43 |         l4.setBackground(new Color(22, 54, 110));
44 |         passTf.setEchoChar('*');
45 |
46 |         // event handling
47 |         loginBtn.addActionListener(this);
48 |         registerBtn.addActionListener(this);
49 |         backBtn.addActionListener(this);
50 |
51 |         // adding components
52 |         add(l1);
53 |         add(l4);
54 |         add(l2);
55 |         add(userTf);
56 |         add(l3);
57 |         add(passTf);
58 |         add(loginBtn);
59 |         add(backBtn);
60 |         if (type.equals("Customer"))
61 |             add(registerBtn); // only customers can register
62 |
```

```
 63 |         // closing the window
 64 |         addWindowListener(this);
 65 |
 66 |         // creating the frame
 67 |         setTitle(type + " Login");
 68 |         setSize(450, 280);
 69 |         setLocationRelativeTo(null);
 70 |         setVisible(true);
 71 |     }
 72 |
 73 |     // event handling method
 74 |     public void actionPerformed(ActionEvent e)
 75 |     {
 76 |         Button btn = (Button) e.getSource();
 77 |         String user = userTf.getText().trim();
 78 |         String pass = passTf.getText();
 79 |
 80 |         if (btn == backBtn)
 81 |             dispose();
 82 |         else if (btn == registerBtn)
 83 |             new RegisterFrame();
 84 |         else if (user.isEmpty() || pass.isEmpty())
 85 |             new MessageBox(this, "Please fill both the fields");
 86 |         else
 87 |         {
 88 |             try
 89 |             {
 90 |                 Connection con = DBConnection.getConnection();
 91 |                 String sql = "SELECT * FROM customers WHERE email = ?";
 92 |                 if (type.equals("Admin"))
 93 |                     sql = "SELECT * FROM admins WHERE username = ?";
 94 |                 PreparedStatement ps = con.prepareStatement(sql);
 95 |                 ps.setString(1, user);
 96 |                 ResultSet rs = ps.executeQuery();
 97 |
 98 |                 // password is checked in java so that it is case sensitive
 99 |                 if (!rs.next() || !rs.getString("password").equals(pass))
100 |                     new MessageBox(this, "Wrong login details");
101 |                 else
102 |                 {
103 |                     if (type.equals("Admin"))
104 |                         new AdminDashboard(user);
105 |                     else
106 |                         new CustomerDashboard(rs.getInt("customer_id"), rs.getString("name"));
107 |                     dispose();
108 |                 }
109 |                 con.close();
110 |             }
111 |             catch (Exception ex)
112 |             {
113 |                 System.out.println(ex);
114 |                 new MessageBox(this, "Error: " + ex.getMessage());
115 |             }
116 |         }
117 |     }
118 |
119 |     // window events (only closing is used, WindowListener needs all 7 methods)
120 |     public void windowClosing(WindowEvent we)
121 |     {
122 |         dispose();
123 |     }
124 |     public void windowOpened(WindowEvent we) {}
125 |     public void windowClosed(WindowEvent we) {}
126 |     public void windowIconified(WindowEvent we) {}
127 |     public void windowDeiconified(WindowEvent we) {}
128 |     public void windowActivated(WindowEvent we) {}
129 |     public void windowDeactivated(WindowEvent we) {}
130 | }
```

src/RegisterFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | public class RegisterFrame extends Frame implements ActionListener, WindowListener
 6 | {
 7 |     Label l1, l2, l3, l4, l5, l6, l7;
 8 |     TextField nameTf, emailTf, phoneTf, passTf, confirmTf;
 9 |     Button registerBtn, backBtn;
10 |
11 |     public RegisterFrame()
12 |     {
13 |         // setting the layout (2 columns, like label and text field)
14 |         setLayout(new GridLayout(7, 2, 0, 10));
15 |
16 |         // creating objects
17 |         l1 = new Label("   CREATE ACCOUNT");
18 |         l7 = new Label("");
19 |         l2 = new Label("    Full Name");
20 |         l3 = new Label("    Email");
21 |         l4 = new Label("    Phone (10 digits)");
22 |         l5 = new Label("    Password");
23 |         l6 = new Label("    Confirm Password");
24 |         nameTf = new TextField(20);
25 |         emailTf = new TextField(20);
26 |         phoneTf = new TextField(20);
27 |         passTf = new TextField(20);
28 |         confirmTf = new TextField(20);
29 |         registerBtn = new Button("Register");
30 |         backBtn = new Button("Back");
31 |
32 |         // design
33 |         setBackground(new Color(232, 240, 250));
34 |         setFont(new Font("Arial", Font.PLAIN, 15));
35 |         l1.setBackground(new Color(22, 54, 110));
36 |         l1.setForeground(Color.white);
37 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
38 |         l7.setBackground(new Color(22, 54, 110));
39 |         passTf.setEchoChar('*');
40 |         confirmTf.setEchoChar('*');
41 |
42 |         // event handling
43 |         registerBtn.addActionListener(this);
44 |         backBtn.addActionListener(this);
45 |
46 |         // adding components
47 |         add(l1);
48 |         add(l7);
49 |         add(l2);
50 |         add(nameTf);
51 |         add(l3);
52 |         add(emailTf);
53 |         add(l4);
54 |         add(phoneTf);
55 |         add(l5);
56 |         add(passTf);
57 |         add(l6);
58 |         add(confirmTf);
59 |         add(registerBtn);
60 |         add(backBtn);
61 |
62 |         // closing the window
63 |         addWindowListener(this);
64 |
65 |         // creating the frame
66 |         setTitle("Customer Registration");
67 |         setSize(480, 450);
68 |         setLocationRelativeTo(null);
69 |         setVisible(true);
70 |     }
71 |
72 |     // event handling method
```

```
 73 |     public void actionPerformed(ActionEvent e)
 74 |     {
 75 |         Button btn = (Button) e.getSource();
 76 |         String name = nameTf.getText().trim();
 77 |         String email = emailTf.getText().trim();
 78 |         String phone = phoneTf.getText().trim();
 79 |         String pass = passTf.getText();
 80 |
 81 |         // checking the inputs
 82 |         if (btn == backBtn)
 83 |             dispose();
 84 |         else if (name.isEmpty() || email.isEmpty() || phone.isEmpty() || pass.isEmpty())
 85 |             new MessageBox(this, "All fields are required");
 86 |         else if (!email.matches("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}"))
 87 |             new MessageBox(this, "Please enter a valid email");
 88 |         else if (!phone.matches("[0-9]{10}"))
 89 |             new MessageBox(this, "Phone number must be 10 digits");
 90 |         else if (pass.length() < 4)
 91 |             new MessageBox(this, "Password must be at least 4 characters");
 92 |         else if (!pass.equals(confirmTf.getText()))
 93 |             new MessageBox(this, "Passwords do not match");
 94 |         else
 95 |         {
 96 |             try
 97 |             {
 98 |                 Connection con = DBConnection.getConnection();
 99 |                 PreparedStatement ps = con.prepareStatement("INSERT INTO customers (name, email,
    > phone, password) VALUES (?, ?, ?, ?)");
100 |                 ps.setString(1, name);
101 |                 ps.setString(2, email);
102 |                 ps.setString(3, phone);
103 |                 ps.setString(4, pass);
104 |                 ps.executeUpdate();
105 |                 con.close();
106 |                 new MessageBox(this, "Registration successful! You can login now.");
107 |                 dispose();
108 |             }
109 |             catch (Exception ex)
110 |             {
111 |                 System.out.println(ex);
112 |                 new MessageBox(this, "Email already registered or error: " + ex.getMessage());
113 |             }
114 |         }
115 |     }
116 |
117 |     // window events (only closing is used, WindowListener needs all 7 methods)
118 |     public void windowClosing(WindowEvent we)
119 |     {
120 |         dispose();
121 |     }
122 |     public void windowOpened(WindowEvent we) {}
123 |     public void windowClosed(WindowEvent we) {}
124 |     public void windowIconified(WindowEvent we) {}
125 |     public void windowDeiconified(WindowEvent we) {}
126 |     public void windowActivated(WindowEvent we) {}
127 |     public void windowDeactivated(WindowEvent we) {}
128 | }
```

src/AdminDashboard.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 |
 4 | public class AdminDashboard extends Frame implements ActionListener, WindowListener
 5 | {
 6 |     Label l1, l2;
 7 |     Button manageBtn, bookingsBtn, logoutBtn;
 8 |
 9 |     public AdminDashboard(String adminName)
10 |     {
11 |         // setting the layout
12 |         setLayout(new GridLayout(5, 1, 10, 10));
13 |
14 |         // creating objects
15 |         l1 = new Label("ADMIN DASHBOARD", Label.CENTER);
16 |         l2 = new Label("Welcome, " + adminName, Label.CENTER);
17 |         manageBtn = new Button("Manage Flights");
18 |         bookingsBtn = new Button("View Bookings");
19 |         logoutBtn = new Button("Logout");
20 |
21 |         // design
22 |         setBackground(new Color(232, 240, 250));
23 |         setFont(new Font("Arial", Font.PLAIN, 15));
24 |         l1.setBackground(new Color(22, 54, 110));
25 |         l1.setForeground(Color.white);
26 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
27 |         l2.setFont(new Font("Arial", Font.BOLD, 15));
28 |
29 |         // event handling
30 |         manageBtn.addActionListener(this);
31 |         bookingsBtn.addActionListener(this);
32 |         logoutBtn.addActionListener(this);
33 |
34 |         // adding components
35 |         add(l1);
36 |         add(l2);
37 |         add(manageBtn);
38 |         add(bookingsBtn);
39 |         add(logoutBtn);
40 |
41 |         // closing the window
42 |         addWindowListener(this);
43 |
44 |         // creating the frame
45 |         setTitle("Admin Dashboard");
46 |         setSize(420, 360);
47 |         setLocationRelativeTo(null);
48 |         setVisible(true);
49 |     }
50 |
51 |     // event handling method
52 |     public void actionPerformed(ActionEvent e)
53 |     {
54 |         Button btn = (Button) e.getSource();
55 |
56 |         if (btn == manageBtn)
57 |             new ManageFlightsFrame();
58 |         else if (btn == bookingsBtn)
59 |             new ViewBookingsFrame();
60 |         else
61 |         {
62 |             new LoginFrame("Admin");
63 |             dispose();
64 |         }
65 |     }
66 |
67 |     // window events (only closing is used, WindowListener needs all 7 methods)
68 |     public void windowClosing(WindowEvent we)
69 |     {
70 |         dispose();
71 |     }
72 |     public void windowOpened(WindowEvent we) {}
```

```
    public void windowClosed(WindowEvent we) {}
    public void windowIconified(WindowEvent we) {}
    public void windowDeiconified(WindowEvent we) {}
    public void windowActivated(WindowEvent we) {}
    public void windowDeactivated(WindowEvent we) {}
}
```

src/CustomerDashboard.java

```
import java.awt.*;
import java.awt.event.*;

public class CustomerDashboard extends Frame implements ActionListener, WindowListener
{
    Label l1, l2;
    Button flightsBtn, bookingsBtn, logoutBtn;
    int customerId; // customer who logged in

    public CustomerDashboard(int id, String name)
    {
        customerId = id;

        // setting the layout
        setLayout(new GridLayout(5, 1, 10, 10));

        // creating objects
        l1 = new Label("CUSTOMER DASHBOARD", Label.CENTER);
        l2 = new Label("Welcome, " + name, Label.CENTER);
        flightsBtn = new Button("View Flights & Book Ticket");
        bookingsBtn = new Button("My Bookings");
        logoutBtn = new Button("Logout");

        // design
        setBackground(new Color(232, 240, 250));
        setFont(new Font("Arial", Font.PLAIN, 15));
        l1.setBackground(new Color(22, 54, 110));
        l1.setForeground(Color.white);
        l1.setFont(new Font("Arial", Font.BOLD, 22));
        l2.setFont(new Font("Arial", Font.BOLD, 15));

        // event handling
        flightsBtn.addActionListener(this);
        bookingsBtn.addActionListener(this);
        logoutBtn.addActionListener(this);

        // adding components
        add(l1);
        add(l2);
        add(flightsBtn);
        add(bookingsBtn);
        add(logoutBtn);

        // closing the window
        addWindowListener(this);

        // creating the frame
        setTitle("Customer Dashboard");
        setSize(420, 360);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    // event handling method
    public void actionPerformed(ActionEvent e)
    {
        Button btn = (Button) e.getSource();

        if (btn == flightsBtn)
            new AvailableFlightsFrame(customerId);
        else if (btn == bookingsBtn)
            new MyBookingsFrame(customerId);
        else
```

```
64 |         {
65 |             new LoginFrame("Customer");
66 |             dispose();
67 |         }
68 |     }
69 |
70 |     // window events (only closing is used, WindowListener needs all 7 methods)
71 |     public void windowClosing(WindowEvent we)
72 |     {
73 |         dispose();
74 |     }
75 |     public void windowOpened(WindowEvent we) {}
76 |     public void windowClosed(WindowEvent we) {}
77 |     public void windowIconified(WindowEvent we) {}
78 |     public void windowDeiconified(WindowEvent we) {}
79 |     public void windowActivated(WindowEvent we) {}
80 |     public void windowDeactivated(WindowEvent we) {}
81 | }
```

src/ManageFlightsFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // view all flights, search, add, update and delete flights
 6 | public class ManageFlightsFrame extends Frame implements ActionListener, WindowListener
 7 | {
 8 |     Label l1, l2, l3;
 9 |     TextField searchTf;
10 |     List flightList;
11 |     Button searchBtn, showAllBtn, addBtn, updateBtn, deleteBtn, backBtn;
12 |     Panel p1, p2, p3;
13 |     int[] flightIds = new int[500]; // id of every row in the list
14 |     int total = 0;                  // number of rows in the list
15 |
16 |     public ManageFlightsFrame()
17 |     {
18 |         // setting the layout (no layout manager, every component gets x, y, width, height)
19 |         setLayout(new GridLayout(2, 1, 0, 5)); // top part with buttons, bottom part with the list
20 |
21 |         // creating objects
22 |         l1 = new Label("MANAGE FLIGHTS", Label.CENTER);
23 |         l2 = new Label("Flight No / ID");
24 |         l3 = new Label(String.format(" %-4s %-7s %-10s %-22s %-17s %-11s %s", "ID", "Flight",
   > "Airline", "Route", "Departure", "Price(Rs)", "Seats Left"));
25 |         searchTf = new TextField(10);
26 |         flightList = new List(14);
27 |         searchBtn = new Button("Search");
28 |         showAllBtn = new Button("List All");
29 |         addBtn = new Button("Add Flight");
30 |         updateBtn = new Button("Update Selected");
31 |         deleteBtn = new Button("Delete Selected");
32 |         backBtn = new Button("Back");
33 |
34 |         p1 = new Panel(new GridLayout(4, 1, 5, 8));
35 |         p2 = new Panel(new GridLayout(1, 4, 10, 0));
36 |         p3 = new Panel(new GridLayout(1, 4, 10, 0));
37 |         // design
38 |         setBackground(new Color(232, 240, 250));
39 |         setFont(new Font("Arial", Font.PLAIN, 15));
40 |         l1.setBackground(new Color(22, 54, 110));
41 |         l1.setForeground(Color.white);
42 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
43 |         l3.setFont(new Font("Monospaced", Font.BOLD, 14));
44 |         flightList.setFont(new Font("Monospaced", Font.PLAIN, 14));
45 |
46 |         // event handling
47 |         searchBtn.addActionListener(this);
48 |         showAllBtn.addActionListener(this);
49 |         addBtn.addActionListener(this);
50 |         updateBtn.addActionListener(this);
```

```
51 |         deleteBtn.addActionListener(this);
52 |         backBtn.addActionListener(this);
53 |
54 |         // adding components
55 |         p2.add(l2);
56 |         p2.add(searchTf);
57 |         p2.add(searchBtn);
58 |         p2.add(showAllBtn);
59 |         p3.add(addBtn);
60 |         p3.add(updateBtn);
61 |         p3.add(deleteBtn);
62 |         p3.add(backBtn);
63 |         p1.add(l1);
64 |         p1.add(p2);
65 |         p1.add(p3);
66 |         p1.add(l3);
67 |         add(p1);
68 |         add(flightList);
69 |
70 |         // closing the window
71 |         addWindowListener(this);
72 |
73 |         // creating the frame
74 |         setTitle("Manage Flights");
75 |         setSize(900, 600);
76 |         setLocationRelativeTo(null);
77 |         setVisible(true);
78 |
79 |         loadFlights("");
80 |     }
81 |
82 |     // shows flights in the list, if search is empty it shows all flights
83 |     public void loadFlights(String search)
84 |     {
85 |         flightList.removeAll();
86 |         total = 0;
87 |         try
88 |         {
89 |             Connection con = DBConnection.getConnection();
90 |             PreparedStatement ps = con.prepareStatement("SELECT * FROM flights WHERE ? = '' OR
   > flight_no = ? OR flight_id = ? ORDER BY departure_time");
91 |             ps.setString(1, search);
92 |             ps.setString(2, search);
93 |             ps.setString(3, search);
94 |             ResultSet rs = ps.executeQuery();
95 |             while (rs.next() && total < 500)
96 |             {
97 |                 flightIds[total] = rs.getInt("flight_id");
98 |                 total++;
99 |                 flightList.add(String.format("%-4d %-7s %-10s %-22s %-17s %-11.2f %d/%d",
   > rs.getInt("flight_id"), rs.getString("flight_no"),
100 |                         rs.getString("airline"), rs.getString("source") + " -> " +
   > rs.getString("destination"),
101 |                         rs.getString("departure_time").substring(0, 16), rs.getDouble("price"),
   > rs.getInt("available_seats"), rs.getInt("total_seats")));
102 |             }
103 |             con.close();
104 |             if (total == 0)
105 |                 new MessageBox(this, "No flight found");
106 |         }
107 |         catch (Exception e)
108 |         {
109 |             System.out.println(e);
110 |             new MessageBox(this, "Error: " + e.getMessage());
111 |         }
112 |     }
113 |
114 |     // event handling method
115 |     public void actionPerformed(ActionEvent e)
116 |     {
117 |         Button btn = (Button) e.getSource();
118 |         int i = flightList.getSelectedIndex();
119 |
120 |         if (btn == backBtn)
```

```
121 |             dispose();
122 |         else if (btn == searchBtn)
123 |             loadFlights(searchTf.getText().trim().toUpperCase());
124 |         else if (btn == showAllBtn)
125 |         {
126 |             searchTf.setText("");
127 |             loadFlights("");
128 |         }
129 |         else if (btn == addBtn)
130 |             new FlightFormFrame(this, 0); // 0 means new flight
131 |         else if (i < 0)
132 |             new MessageBox(this, "Please select a flight from the list first");
133 |         else if (btn == updateBtn)
134 |             new FlightFormFrame(this, flightIds[i]);
135 |         else if (new MessageBox(this, "Delete flight ID " + flightIds[i] + "?", true).yes)
136 |         {
137 |             try
138 |             {
139 |                 // deletes only if there are no confirmed bookings (seats are deleted by ON DELETE
    > CASCADE)
140 |                 Connection con = DBConnection.getConnection();
141 |                 PreparedStatement ps = con.prepareStatement("DELETE FROM flights WHERE flight_id = ?
    > AND NOT EXISTS "
142 |                         + "(SELECT * FROM bookings WHERE flight_id = ? AND status = 'CONFIRMED')");
143 |                 ps.setInt(1, flightIds[i]);
144 |                 ps.setInt(2, flightIds[i]);
145 |                 if (ps.executeUpdate() > 0)
146 |                     new MessageBox(this, "Flight deleted successfully");
147 |                 else
148 |                     new MessageBox(this, "Cannot delete. This flight has active bookings");
149 |                 con.close();
150 |                 loadFlights("");
151 |             }
152 |             catch (Exception ex)
153 |             {
154 |                 System.out.println(ex);
155 |                 new MessageBox(this, "Error: " + ex.getMessage());
156 |             }
157 |         }
158 |     }
159 |
160 |     // window events (only closing is used, WindowListener needs all 7 methods)
161 |     public void windowClosing(WindowEvent we)
162 |     {
163 |         dispose();
164 |     }
165 |     public void windowOpened(WindowEvent we) {}
166 |     public void windowClosed(WindowEvent we) {}
167 |     public void windowIconified(WindowEvent we) {}
168 |     public void windowDeiconified(WindowEvent we) {}
169 |     public void windowActivated(WindowEvent we) {}
170 |     public void windowDeactivated(WindowEvent we) {}
171 | }
```

src/FlightFormFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 | import java.time.*;
 5 |
 6 | // same window is used to add a new flight (flightId = 0) and to update a flight
 7 | public class FlightFormFrame extends Frame implements ActionListener, WindowListener
 8 | {
 9 |     Label l1, l2, l3, l4, l5, l6, l7, l8, l9, l10;
10 |     TextField noTf, airlineTf, sourceTf, destTf, depTf, arrTf, priceTf, seatsTf;
11 |     Button saveBtn, backBtn;
12 |     ManageFlightsFrame parent; // to refresh the list after saving
13 |     int flightId;
14 |
15 |     public FlightFormFrame(ManageFlightsFrame mf, int id)
16 |     {
17 |         parent = mf;
18 |         flightId = id;
19 |
20 |         // setting the layout (2 columns, like label and text field)
21 |         setLayout(new GridLayout(10, 2, 0, 8));
22 |
23 |         // creating objects
24 |         if (flightId == 0)
25 |             l1 = new Label("   ADD NEW FLIGHT");
26 |         else
27 |             l1 = new Label("   UPDATE FLIGHT  (ID " + flightId + ")");
28 |         l10 = new Label("");
29 |         l2 = new Label("    Flight Number");
30 |         l3 = new Label("    Airline");
31 |         l4 = new Label("    Source");
32 |         l5 = new Label("    Destination");
33 |         l6 = new Label("    Departure (yyyy-MM-dd HH:mm)");
34 |         l7 = new Label("    Arrival (yyyy-MM-dd HH:mm)");
35 |         l8 = new Label("    Ticket Price (Rs)");
36 |         l9 = new Label("    Total Seats (max 180)");
37 |         noTf = new TextField(18);
38 |         airlineTf = new TextField(18);
39 |         sourceTf = new TextField(18);
40 |         destTf = new TextField(18);
41 |         depTf = new TextField(18);
42 |         arrTf = new TextField(18);
43 |         priceTf = new TextField(18);
44 |         seatsTf = new TextField(18);
45 |         saveBtn = new Button("Save Flight");
46 |         backBtn = new Button("Back");
47 |
48 |         // design
49 |         setBackground(new Color(232, 240, 250));
50 |         setFont(new Font("Arial", Font.PLAIN, 15));
51 |         l1.setBackground(new Color(22, 54, 110));
52 |         l1.setForeground(Color.white);
53 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
54 |         l10.setBackground(new Color(22, 54, 110));
55 |
56 |         // event handling
57 |         saveBtn.addActionListener(this);
58 |         backBtn.addActionListener(this);
59 |
60 |         // adding components
61 |         add(l1);
62 |         add(l10);
63 |         add(l2);
64 |         add(noTf);
65 |         add(l3);
66 |         add(airlineTf);
67 |         add(l4);
68 |         add(sourceTf);
69 |         add(l5);
70 |         add(destTf);
71 |         add(l6);
72 |         add(depTf);
```

```
 73 |         add(l7);
 74 |         add(arrTf);
 75 |         add(l8);
 76 |         add(priceTf);
 77 |         add(l9);
 78 |         add(seatsTf);
 79 |         add(saveBtn);
 80 |         add(backBtn);
 81 |
 82 |         // closing the window
 83 |         addWindowListener(this);
 84 |
 85 |         // creating the frame
 86 |         setTitle("Flight Details");
 87 |         setSize(600, 560);
 88 |         setLocationRelativeTo(null);
 89 |         setVisible(true);
 90 |
 91 |         // update: showing the old values of the flight
 92 |         if (flightId != 0)
 93 |         {
 94 |             try
 95 |             {
 96 |                 Connection con = DBConnection.getConnection();
 97 |                 PreparedStatement ps = con.prepareStatement("SELECT * FROM flights WHERE flight_id =
    > ?");
 98 |                 ps.setInt(1, flightId);
 99 |                 ResultSet rs = ps.executeQuery();
100 |                 rs.next();
101 |                 noTf.setText(rs.getString("flight_no"));
102 |                 airlineTf.setText(rs.getString("airline"));
103 |                 sourceTf.setText(rs.getString("source"));
104 |                 destTf.setText(rs.getString("destination"));
105 |                 depTf.setText(rs.getString("departure_time").substring(0, 16));
106 |                 arrTf.setText(rs.getString("arrival_time").substring(0, 16));
107 |                 priceTf.setText("" + rs.getDouble("price"));
108 |                 seatsTf.setText("" + rs.getInt("total_seats"));
109 |                 con.close();
110 |             }
111 |             catch (Exception e)
112 |             {
113 |                 System.out.println(e);
114 |             }
115 |         }
116 |     }
117 |
118 |     // event handling method
119 |     public void actionPerformed(ActionEvent e)
120 |     {
121 |         Button btn = (Button) e.getSource();
122 |         if (btn == backBtn)
123 |         {
124 |             dispose();
125 |             return;
126 |         }
127 |
128 |         // checking the values
129 |         LocalDateTime dep, arr;
130 |         double price;
131 |         int seats;
132 |         try
133 |         {
134 |             dep = LocalDateTime.parse(depTf.getText().trim().replace(" ", "T"));
135 |             arr = LocalDateTime.parse(arrTf.getText().trim().replace(" ", "T"));
136 |             price = Double.parseDouble(priceTf.getText().trim());
137 |             seats = Integer.parseInt(seatsTf.getText().trim());
138 |         }
139 |         catch (Exception ex)
140 |         {
141 |             new MessageBox(this, "Please enter correct values\nTime format: yyyy-MM-dd HH:mm\nPrice
    > and seats must be numbers");
142 |             return;
143 |         }
144 |
```

```
145 |         if (noTf.getText().trim().isEmpty() || airlineTf.getText().trim().isEmpty() ||
    > sourceTf.getText().trim().isEmpty() || destTf.getText().trim().isEmpty())
146 |             new MessageBox(this, "All fields are required");
147 |         else if (sourceTf.getText().trim().equalsIgnoreCase(destTf.getText().trim()))
148 |             new MessageBox(this, "Source and destination cannot be same");
149 |         else if (!arr.isAfter(dep))
150 |             new MessageBox(this, "Arrival time must be after departure time");
151 |         else if (price <= 0 || seats < 1 || seats > 180)
152 |             new MessageBox(this, "Price must be more than 0 and seats between 1 and 180");
153 |         else if (flightId == 0 && dep.isBefore(LocalDateTime.now()))
154 |             new MessageBox(this, "Departure time must be in the future");
155 |         else
156 |             saveFlight(price, seats);
157 |     }
158 |
159 |     // adds or updates the flight in one transaction, seats are made again if total seats is changed
160 |     void saveFlight(double price, int seats)
161 |     {
162 |         Connection con = null;
163 |         try
164 |         {
165 |             con = DBConnection.getConnection();
166 |             con.setAutoCommit(false); // start transaction
167 |             int available = seats;
168 |             boolean makeSeats = true;
169 |
170 |             if (flightId != 0)
171 |             {
172 |                 // lock the flight row, seats can be changed only when there are no confirmed bookings
173 |                 PreparedStatement ps1 = con.prepareStatement("SELECT total_seats, available_seats,
    > (SELECT COUNT(*) FROM bookings "
174 |                         + "WHERE flight_id = ? AND status = 'CONFIRMED') AS booked FROM flights WHERE
    > flight_id = ? FOR UPDATE");
175 |                 ps1.setInt(1, flightId);
176 |                 ps1.setInt(2, flightId);
177 |                 ResultSet rs = ps1.executeQuery();
178 |                 rs.next();
179 |                 if (seats == rs.getInt("total_seats"))
180 |                 {
181 |                     available = rs.getInt("available_seats");
182 |                     makeSeats = false;
183 |                 }
184 |                 else if (rs.getInt("booked") > 0)
185 |                     throw new Exception("Total seats cannot be changed, this flight has active
    > bookings");
186 |                 else
187 |                 {
188 |                     PreparedStatement ps2 = con.prepareStatement("DELETE FROM seats WHERE flight_id =
    > ?");
189 |                     ps2.setInt(1, flightId);
190 |                     ps2.executeUpdate();
191 |                 }
192 |             }
193 |
194 |             String sql = "UPDATE flights SET flight_no = ?, airline = ?, source = ?, destination = ?,
    > departure_time = ?, "
195 |                     + "arrival_time = ?, price = ?, total_seats = ?, available_seats = ? WHERE
    > flight_id = ?";
196 |             if (flightId == 0)
197 |                 sql = "INSERT INTO flights (flight_no, airline, source, destination, departure_time,
    > arrival_time, price, "
198 |                         + "total_seats, available_seats) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
199 |             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
200 |             ps.setString(1, noTf.getText().trim().toUpperCase());
201 |             ps.setString(2, airlineTf.getText().trim());
202 |             ps.setString(3, sourceTf.getText().trim());
203 |             ps.setString(4, destTf.getText().trim());
204 |             ps.setString(5, depTf.getText().trim());
205 |             ps.setString(6, arrTf.getText().trim());
206 |             ps.setDouble(7, price);
207 |             ps.setInt(8, seats);
208 |             ps.setInt(9, available);
209 |             if (flightId != 0)
210 |                 ps.setInt(10, flightId);
```

```
211 |             ps.executeUpdate();
212 |
213 |             // new flight: flight_id is AUTO_INCREMENT, so we get it back from MySQL
214 |             int id = flightId;
215 |             ResultSet keys = ps.getGeneratedKeys();
216 |             if (flightId == 0 && keys.next())
217 |                 id = keys.getInt(1);
218 |
219 |             // making seats like 1A, 1B ... 1F, 2A ... (6 seats in a row)
220 |             PreparedStatement ps3 = con.prepareStatement("INSERT INTO seats (flight_id, seat_no)
    > VALUES (?, ?)");
221 |             for (int i = 0; makeSeats && i < seats; i++)
222 |             {
223 |                 ps3.setInt(1, id);
224 |                 ps3.setString(2, (i / 6 + 1) + "" + "ABCDEF".charAt(i % 6));
225 |                 ps3.executeUpdate();
226 |             }
227 |
228 |             con.commit(); // save everything
229 |             con.close();
230 |             new MessageBox(this, "Flight saved successfully!\nFlight ID: " + id);
231 |             parent.loadFlights("");
232 |             dispose();
233 |         }
234 |         catch (Exception e)
235 |         {
236 |             // undo all changes
237 |             try
238 |             {
239 |                 con.rollback();
240 |                 con.close();
241 |             }
242 |             catch (Exception ex)
243 |             {
244 |                 System.out.println(ex);
245 |             }
246 |             new MessageBox(this, "Error: " + e.getMessage());
247 |         }
248 |     }
249 |
250 |     // window events (only closing is used, WindowListener needs all 7 methods)
251 |     public void windowClosing(WindowEvent we)
252 |     {
253 |         dispose();
254 |     }
255 |     public void windowOpened(WindowEvent we) {}
256 |     public void windowClosed(WindowEvent we) {}
257 |     public void windowIconified(WindowEvent we) {}
258 |     public void windowDeiconified(WindowEvent we) {}
259 |     public void windowActivated(WindowEvent we) {}
260 |     public void windowDeactivated(WindowEvent we) {}
261 | }
```

src/AvailableFlightsFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // shows all flights added by admin, customer selects one and books it
 6 | public class AvailableFlightsFrame extends Frame implements ActionListener, WindowListener
 7 | {
 8 |     Label l1, l2, l3, l4;
 9 |     Choice fromChoice, toChoice;
10 |     List flightList;
11 |     Button filterBtn, showAllBtn, bookBtn, backBtn;
12 |     Panel p1, p2, p3;
13 |     int customerId;
14 |     int[] flightIds = new int[500];     // id of every row in the list
15 |     String[] status = new String[500];  // Available / Full / Departed of every row
16 |     int total = 0;                      // number of rows in the list
17 |
18 |     public AvailableFlightsFrame(int id)
19 |     {
20 |         customerId = id;
21 |
22 |         // setting the layout (no layout manager, every component gets x, y, width, height)
23 |         setLayout(new GridLayout(2, 1, 0, 5)); // top part with filter and buttons, bottom part with
   > the list
24 |
25 |         // creating objects
26 |         l1 = new Label("AVAILABLE FLIGHTS", Label.CENTER);
27 |         l2 = new Label("From", Label.RIGHT);
28 |         l3 = new Label("To", Label.RIGHT);
29 |         l4 = new Label(String.format(" %-4s %-7s %-10s %-22s %-17s %-11s %-11s %s", "ID", "Flight",
   > "Airline", "Route", "Departure", "Price(Rs)", "Seats Left", "Status"));
30 |         fromChoice = new Choice();
31 |         toChoice = new Choice();
32 |         flightList = new List(14);
33 |         filterBtn = new Button("Filter");
34 |         showAllBtn = new Button("List All");
35 |         bookBtn = new Button("Select Flight");
36 |         backBtn = new Button("Back");
37 |
38 |         p1 = new Panel(new GridLayout(4, 1, 5, 8));
39 |         p2 = new Panel(new GridLayout(1, 6, 10, 0));
40 |         p3 = new Panel(new GridLayout(1, 2, 10, 0));
41 |         // design
42 |         setBackground(new Color(232, 240, 250));
43 |         setFont(new Font("Arial", Font.PLAIN, 15));
44 |         l1.setBackground(new Color(22, 54, 110));
45 |         l1.setForeground(Color.white);
46 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
47 |         l4.setFont(new Font("Monospaced", Font.BOLD, 14));
48 |         flightList.setFont(new Font("Monospaced", Font.PLAIN, 14));
49 |         fromChoice.add("All");
50 |         toChoice.add("All");
51 |
52 |         // event handling
53 |         filterBtn.addActionListener(this);
54 |         showAllBtn.addActionListener(this);
55 |         bookBtn.addActionListener(this);
56 |         backBtn.addActionListener(this);
57 |
58 |         // adding components
59 |         p2.add(l2);
60 |         p2.add(fromChoice);
61 |         p2.add(l3);
62 |         p2.add(toChoice);
63 |         p2.add(filterBtn);
64 |         p2.add(showAllBtn);
65 |         p3.add(bookBtn);
66 |         p3.add(backBtn);
67 |         p1.add(l1);
68 |         p1.add(p2);
69 |         p1.add(p3);
70 |         p1.add(l4);
```

```
71 |         add(p1);
72 |         add(flightList);
73 |
74 |         // closing the window
75 |         addWindowListener(this);
76 |
77 |         // creating the frame
78 |         setTitle("Available Flights");
79 |         setSize(1000, 600);
80 |         setLocationRelativeTo(null);
81 |         setVisible(true);
82 |
83 |         // filling both Choice boxes with city names from database
84 |         try
85 |         {
86 |             Connection con = DBConnection.getConnection();
87 |             Statement stmt = con.createStatement();
88 |             ResultSet rs = stmt.executeQuery("SELECT source AS city FROM flights UNION SELECT
   > destination FROM flights ORDER BY city");
89 |             while (rs.next())
90 |             {
91 |                 fromChoice.add(rs.getString("city"));
92 |                 toChoice.add(rs.getString("city"));
93 |             }
94 |             con.close();
95 |         }
96 |         catch (Exception e)
97 |         {
98 |             System.out.println(e);
99 |         }
100 |         loadFlights("All", "All");
101 |     }
102 |
103 |     // shows all flights ("All" means any city), departed flights are shown but cannot be booked
104 |     void loadFlights(String from, String to)
105 |     {
106 |         flightList.removeAll();
107 |         total = 0;
108 |         try
109 |         {
110 |             Connection con = DBConnection.getConnection();
111 |             PreparedStatement ps = con.prepareStatement("SELECT *, departure_time < NOW() AS departed
   > FROM flights "
112 |                     + "WHERE (? = 'All' OR source = ?) AND (? = 'All' OR destination = ?) ORDER BY
   > departure_time");
113 |             ps.setString(1, from);
114 |             ps.setString(2, from);
115 |             ps.setString(3, to);
116 |             ps.setString(4, to);
117 |             ResultSet rs = ps.executeQuery();
118 |             while (rs.next() && total < 500)
119 |             {
120 |                 String st = "Available";
121 |                 if (rs.getInt("departed") == 1)
122 |                     st = "Departed";
123 |                 else if (rs.getInt("available_seats") == 0)
124 |                     st = "Full";
125 |
126 |                 flightIds[total] = rs.getInt("flight_id");
127 |                 status[total] = st;
128 |                 total++;
129 |                 flightList.add(String.format("%-4d %-7s %-10s %-22s %-17s %-11.2f %-11s %s",
   > rs.getInt("flight_id"), rs.getString("flight_no"),
130 |                         rs.getString("airline"), rs.getString("source") + " -> " +
   > rs.getString("destination"), rs.getString("departure_time").substring(0, 16),
131 |                         rs.getDouble("price"), rs.getInt("available_seats") + "/" +
   > rs.getInt("total_seats"), st));
132 |             }
133 |             con.close();
134 |             if (total == 0)
135 |                 new MessageBox(this, "No flights found");
136 |         }
137 |         catch (Exception e)
138 |         {
```

```
139 |             System.out.println(e);
140 |             new MessageBox(this, "Error: " + e.getMessage());
141 |         }
142 |     }
143 |
144 |     // event handling method
145 |     public void actionPerformed(ActionEvent e)
146 |     {
147 |         Button btn = (Button) e.getSource();
148 |         int i = flightList.getSelectedIndex();
149 |
150 |         if (btn == backBtn)
151 |             dispose();
152 |         else if (btn == filterBtn)
153 |             loadFlights(fromChoice.getSelectedItem(), toChoice.getSelectedItem());
154 |         else if (btn == showAllBtn)
155 |         {
156 |             fromChoice.select(0);
157 |             toChoice.select(0);
158 |             loadFlights("All", "All");
159 |         }
160 |         else if (i < 0)
161 |             new MessageBox(this, "Please select a flight from the list");
162 |         else if (!status[i].equals("Available"))
163 |             new MessageBox(this, "Sorry, this flight is " + status[i]);
164 |         else
165 |             new SeatSelectionFrame(this, customerId, flightIds[i]);
166 |     }
167 |
168 |     // window events (only closing is used, WindowListener needs all 7 methods)
169 |     public void windowClosing(WindowEvent we)
170 |     {
171 |         dispose();
172 |     }
173 |     public void windowOpened(WindowEvent we) {}
174 |     public void windowClosed(WindowEvent we) {}
175 |     public void windowIconified(WindowEvent we) {}
176 |     public void windowDeiconified(WindowEvent we) {}
177 |     public void windowActivated(WindowEvent we) {}
178 |     public void windowDeactivated(WindowEvent we) {}
179 | }
```

src/SeatSelectionFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // shows the aeroplane seat map, customer selects one or more seats (max 6)
 6 | public class SeatSelectionFrame extends Frame implements ActionListener, WindowListener
 7 | {
 8 |     Label l1, l2, l3, selectedLabel;
 9 |     Button continueBtn, backBtn;
10 |     Panel p1, p2;
11 |     SeatMap map;
12 |     AvailableFlightsFrame flightsFrame; // to refresh seat count after booking
13 |     int customerId, flightId;
14 |     double price = 0;            // price of one ticket
15 |     int[] seatIds = new int[0];  // seat_id of every seat
16 |
17 |     public SeatSelectionFrame(AvailableFlightsFrame af, int cid, int fid)
18 |     {
19 |         flightsFrame = af;
20 |         customerId = cid;
21 |         flightId = fid;
22 |
23 |         // setting the layout (no layout manager, every component gets x, y, width, height)
24 |         setLayout(new GridLayout(2, 1, 0, 5)); // top part with details and buttons, bottom part with
   > the seat map
25 |
26 |         // creating objects
27 |         l1 = new Label("SELECT YOUR SEATS", Label.CENTER);
28 |         l2 = new Label("", Label.CENTER); // flight details, filled from database
29 |         l3 = new Label("White = Available     Green = Selected     Grey (X) = Booked", Label.CENTER);
30 |         selectedLabel = new Label("", Label.CENTER);
31 |         continueBtn = new Button("Continue");
32 |         backBtn = new Button("Back");
33 |         map = new SeatMap(this);
34 |
35 |         p1 = new Panel(new GridLayout(5, 1, 0, 8));
36 |         p2 = new Panel(new GridLayout(1, 2, 20, 0));
37 |         // design
38 |         setBackground(new Color(232, 240, 250));
39 |         setFont(new Font("Arial", Font.PLAIN, 15));
40 |         l1.setBackground(new Color(22, 54, 110));
41 |         l1.setForeground(Color.white);
42 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
43 |         l2.setFont(new Font("Arial", Font.BOLD, 15));
44 |         selectedLabel.setFont(new Font("Arial", Font.BOLD, 15));
45 |
46 |         // event handling
47 |         continueBtn.addActionListener(this);
48 |         backBtn.addActionListener(this);
49 |
50 |         // adding components
51 |         p2.add(continueBtn);
52 |         p2.add(backBtn);
53 |         p1.add(l1);
54 |         p1.add(l2);
55 |         p1.add(l3);
56 |         p1.add(selectedLabel);
57 |         p1.add(p2);
58 |         add(p1);
59 |         add(map); // seat map is drawn so that it fits, no scrolling
60 |
61 |         // closing the window
62 |         addWindowListener(this);
63 |
64 |         // creating the frame
65 |         setTitle("Select Seats");
66 |         setSize(1150, 800);
67 |         setLocationRelativeTo(null);
68 |         setVisible(true);
69 |
70 |         loadSeats();
71 |     }
```

```
 72 |
 73 |     // reads flight details and seats from database and gives the seats to the seat map
 74 |     public void loadSeats()
 75 |     {
 76 |         try
 77 |         {
 78 |             Connection con = DBConnection.getConnection();
 79 |             PreparedStatement ps1 = con.prepareStatement("SELECT * FROM flights WHERE flight_id = ?");
 80 |             ps1.setInt(1, flightId);
 81 |             ResultSet rs1 = ps1.executeQuery();
 82 |             rs1.next();
 83 |             price = rs1.getDouble("price");
 84 |             l2.setText(rs1.getString("flight_no") + "   |   " + rs1.getString("source") + " -> " +
    > rs1.getString("destination")
 85 |                     + "   |   " + rs1.getString("departure_time").substring(0, 16) + "   |   Rs. " +
    > price + " per seat");
 86 |
 87 |             // arrays are made with the total seats of the flight
 88 |             int n = rs1.getInt("total_seats");
 89 |             seatIds = new int[n];
 90 |             map.seatNos = new String[n];
 91 |             map.status = new String[n];
 92 |             map.selCount = 0;
 93 |
 94 |             PreparedStatement ps2 = con.prepareStatement("SELECT * FROM seats WHERE flight_id = ?
    > ORDER BY seat_id");
 95 |             ps2.setInt(1, flightId);
 96 |             ResultSet rs2 = ps2.executeQuery();
 97 |             for (int i = 0; i < n && rs2.next(); i++)
 98 |             {
 99 |                 seatIds[i] = rs2.getInt("seat_id");
100 |                 map.seatNos[i] = rs2.getString("seat_no");
101 |                 map.status[i] = rs2.getString("status");
102 |             }
103 |             con.close();
104 |         }
105 |         catch (Exception e)
106 |         {
107 |             System.out.println(e);
108 |             new MessageBox(this, "Error: " + e.getMessage());
109 |         }
110 |         map.repaint();
111 |         updateSelected();
112 |     }
113 |
114 |     // called by the seat map when a seat is clicked, shows selected seats and total price
115 |     public void updateSelected()
116 |     {
117 |         String seats = "";
118 |         for (int j = 0; j < map.selCount; j++)
119 |             seats = seats + map.seatNos[map.selected[j]] + " ";
120 |
121 |         if (seats.isEmpty())
122 |             selectedLabel.setText("Selected seats: none   (click on a seat, max 6)");
123 |         else
124 |             selectedLabel.setText("Selected seats: " + seats + "  |   Total: Rs. " + (price *
    > map.selCount));
125 |     }
126 |
127 |     // event handling method
128 |     public void actionPerformed(ActionEvent e)
129 |     {
130 |         Button btn = (Button) e.getSource();
131 |
132 |         if (btn == backBtn)
133 |             dispose();
134 |         else if (map.selCount == 0)
135 |             new MessageBox(this, "Please select at least one seat");
136 |         else
137 |         {
138 |             // Continue: opens the window to enter details of every passenger
139 |             int[] ids = new int[map.selCount];
140 |             String[] nos = new String[map.selCount];
141 |             for (int j = 0; j < map.selCount; j++)
```

```
142 |             {
143 |                 ids[j] = seatIds[map.selected[j]];
144 |                 nos[j] = map.seatNos[map.selected[j]];
145 |             }
146 |             new PassengerDetailsFrame(this, flightsFrame, customerId, flightId, ids, nos, price);
147 |         }
148 |     }
149 |
150 |     // window events (only closing is used, WindowListener needs all 7 methods)
151 |     public void windowClosing(WindowEvent we)
152 |     {
153 |         dispose();
154 |     }
155 |     public void windowOpened(WindowEvent we) {}
156 |     public void windowClosed(WindowEvent we) {}
157 |     public void windowIconified(WindowEvent we) {}
158 |     public void windowDeiconified(WindowEvent we) {}
159 |     public void windowActivated(WindowEvent we) {}
160 |     public void windowDeactivated(WindowEvent we) {}
161 | }
```

src/SeatMap.java

```
  1 | import java.awt.*;
  2 | import java.awt.event.*;
  3 |
  4 | // draws the aeroplane seat map (nose on the left, A B C | aisle | D E F)
  5 | // the drawing is scaled so the full plane fits in the window
  6 | public class SeatMap extends Canvas implements MouseListener
  7 | {
  8 |     SeatSelectionFrame frame;
  9 |     String[] seatNos = new String[0];  // 1A, 1B, ...
 10 |     String[] status = new String[0];   // AVAILABLE or BOOKED
 11 |     int[] selected = new int[6];       // index of selected seats (max 6)
 12 |     int selCount = 0, rows = 0, offX = 0, offY = 0;
 13 |     int seat = 36, gap = 6, aisle = 44, startX = 230, top = 20;
 14 |     double scale = 1;
 15 |
 16 |     public SeatMap(SeatSelectionFrame f)
 17 |     {
 18 |         frame = f;
 19 |         setBackground(new Color(232, 240, 250));
 20 |
 21 |         addMouseListener(this); // clicking a seat selects it
 22 |     }
 23 |
 24 |     // clicking a seat selects it, clicking it again removes it
 25 |     public void mousePressed(MouseEvent e)
 26 |     {
 27 |         // mouse position is changed back to drawing size (because the drawing is scaled)
 28 |         int x = (int) ((e.getX() - offX) / scale);
 29 |         int y = (int) ((e.getY() - offY) / scale);
 30 |
 31 |         for (int i = 0; i < seatNos.length; i++)
 32 |         {
 33 |             boolean inside = x >= seatX(i) && x <= seatX(i) + seat && y >= seatY(i) && y <= seatY(i) +
    >  seat;
 34 |             if (!inside || !status[i].equals("AVAILABLE"))
 35 |                 continue;
 36 |
 37 |             int pos = findSelected(i);
 38 |             if (pos >= 0)
 39 |             {
 40 |                 selected[pos] = selected[selCount - 1]; // last selected seat comes in its place
 41 |                 selCount--;
 42 |             }
 43 |             else if (selCount == 6)
 44 |                 new MessageBox(frame, "You can book maximum 6 seats at a time");
 45 |             else
 46 |             {
 47 |                 selected[selCount] = i;
 48 |                 selCount++;
```

```
 49 |             }
 50 |             repaint();
 51 |             frame.updateSelected();
 52 |         }
 53 |     }
 54 |
 55 |     // other mouse events are not used (MouseListener needs all 5 methods)
 56 |     public void mouseClicked(MouseEvent e) {}
 57 |     public void mouseReleased(MouseEvent e) {}
 58 |     public void mouseEntered(MouseEvent e) {}
 59 |     public void mouseExited(MouseEvent e) {}
 60 |
 61 |     // returns position of seat i in the selected array, or -1
 62 |     int findSelected(int i)
 63 |     {
 64 |         for (int j = 0; j < selCount; j++)
 65 |             if (selected[j] == i)
 66 |                 return j;
 67 |         return -1;
 68 |     }
 69 |
 70 |     // x position of seat i (i / 6 = row number, rows go from left to right)
 71 |     int seatX(int i)
 72 |     {
 73 |         return startX + (i / 6) * (seat + 8);
 74 |     }
 75 |
 76 |     // y position of seat i (i % 6 = A to F, D E F come after the aisle)
 77 |     int seatY(int i)
 78 |     {
 79 |         int y = top + 25 + (i % 6) * (seat + gap);
 80 |         if (i % 6 >= 3)
 81 |             y = y + aisle - gap;
 82 |         return y;
 83 |     }
 84 |
 85 |     // drawing directly on screen flickers, so update() only calls paint()
 86 |     public void update(Graphics g)
 87 |     {
 88 |         paint(g);
 89 |     }
 90 |
 91 |     public void paint(Graphics g)
 92 |     {
 93 |         if (getWidth() == 0)
 94 |             return;
 95 |         rows = (seatNos.length + 5) / 6;
 96 |         int right = startX + rows * (seat + 8) + 45;         // end of the plane body
 97 |         int bottom = top + 6 * seat + 5 * gap + aisle + 45;  // bottom of the plane body
 98 |         int h = bottom - top, midY = (top + bottom) / 2;
 99 |
100 |         // everything is drawn on an image first, then shown (no flicker)
101 |         Image buffer = createImage(getWidth(), getHeight());
102 |         Graphics2D g2 = (Graphics2D) buffer.getGraphics();
103 |         g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
104 |         g2.setColor(getBackground());
105 |         g2.fillRect(0, 0, getWidth(), getHeight());
106 |
107 |         // scale so the full plane fits in the window
108 |         scale = Math.min(1, Math.min((getWidth() - 20.0) / (right + 20), (getHeight() - 10.0) /
    > (bottom + 20)));
109 |         offX = (int) ((getWidth() - (right + 20) * scale) / 2);
110 |         offY = (int) ((getHeight() - (bottom + 20) * scale) / 2);
111 |         g2.translate(offX, offY);
112 |         g2.scale(scale, scale);
113 |
114 |         // body with round nose (left) and round tail (right)
115 |         g2.setColor(new Color(250, 251, 253));
116 |         g2.fillOval(10, top, 300, h);
117 |         g2.fillOval(right - 150, top, 150, h);
118 |         g2.fillRect(160, top, right - 235, h);
119 |         g2.setColor(new Color(160, 172, 190));
120 |         g2.setStroke(new BasicStroke(2));
121 |         g2.drawArc(10, top, 300, h, 90, 180);
```

```
122 |         g2.drawArc(right - 150, top, 150, h, 270, 180);
123 |         g2.drawLine(160, top, right - 75, top);
124 |         g2.drawLine(160, bottom, right - 75, bottom);
125 |
126 |         // cockpit windows, FRONT, seat letters and row numbers
127 |         g2.setColor(new Color(22, 54, 110));
128 |         g2.fillRoundRect(40, midY - 36, 16, 32, 10, 10);
129 |         g2.fillRoundRect(40, midY + 4, 16, 32, 10, 10);
130 |         g2.setColor(Color.gray);
131 |         g2.setFont(new Font("Arial", Font.BOLD, 13));
132 |         g2.drawString("FRONT", 90, midY + 5);
133 |         for (int c = 0; c < 6; c++)
134 |             g2.drawString("" + "ABCDEF".charAt(c), startX - 22, seatY(c) + 23);
135 |         for (int r = 0; r < rows; r++)
136 |             g2.drawString("" + (r + 1), seatX(r * 6) + 10, (seatY(2) + seat + seatY(3)) / 2 + 5);
137 |
138 |         // seats: white = available, green = selected, grey with X = booked
139 |         g2.setFont(new Font("Arial", Font.BOLD, 14));
140 |         for (int i = 0; i < seatNos.length; i++)
141 |         {
142 |             String text = seatNos[i].substring(seatNos[i].length() - 1);
143 |             Color fill = Color.white;
144 |             Color textColor = Color.white;
145 |             if (findSelected(i) >= 0)
146 |                 fill = new Color(40, 160, 90);
147 |             else if (!status[i].equals("AVAILABLE"))
148 |             {
149 |                 fill = new Color(200, 204, 211);
150 |                 text = "X";
151 |             }
152 |             else
153 |                 textColor = new Color(22, 54, 110);
154 |
155 |             g2.setColor(fill);
156 |             g2.fillRoundRect(seatX(i), seatY(i), seat, seat, 12, 12);
157 |             g2.setColor(new Color(150, 160, 178));
158 |             g2.drawRoundRect(seatX(i), seatY(i), seat, seat, 12, 12);
159 |             g2.setColor(textColor);
160 |             g2.drawString(text, seatX(i) + 13, seatY(i) + 23);
161 |         }
162 |
163 |         g.drawImage(buffer, 0, 0, this);
164 |         g2.dispose();
165 |     }
166 | }
```

src/PassengerDetailsFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // customer enters details of all passengers in one window (one row for every seat)
 6 | // all passengers get the same PNR
 7 | public class PassengerDetailsFrame extends Frame implements ActionListener, WindowListener
 8 | {
 9 |     Label l1, l2, l3, l4, l5, l6, l7, l8;
10 |     Button payBtn, backBtn;
11 |     SeatSelectionFrame seatFrame;
12 |     AvailableFlightsFrame flightsFrame;
13 |     int customerId, flightId, count;
14 |     int[] seatIds;       // seat_id of every selected seat
15 |     String[] seatNos;    // seat number of every selected seat
16 |     double price;        // price of one ticket
17 |     String[] names;      // details of every passenger (read from the rows)
18 |     int[] age;
19 |     String[] genders;
20 |
21 |     public PassengerDetailsFrame(SeatSelectionFrame sf, AvailableFlightsFrame af, int cid, int fid,
   > int[] ids, String[] nos, double pr)
22 |     {
23 |         seatFrame = sf;
24 |         flightsFrame = af;
25 |         customerId = cid;
26 |         flightId = fid;
27 |         seatIds = ids;
28 |         seatNos = nos;
29 |         price = pr;
30 |         count = ids.length; // number of passengers
31 |         names = new String[count];
32 |         age = new int[count];
33 |         genders = new String[count];
34 |
35 |         // setting the layout (4 columns: seat, name, age, gender)
36 |         // rows: title, headings, one row for every passenger, buttons
37 |         setLayout(new GridLayout(count + 3, 4, 0, 8));
38 |
39 |         // creating objects
40 |         l1 = new Label("   PASSENGERS");
41 |         l2 = new Label(count + " passenger(s)");
42 |         l3 = new Label("    Seat");
43 |         l4 = new Label("Full Name");
44 |         l5 = new Label("Age");
45 |         l6 = new Label("Gender");
46 |         l7 = new Label("Total: Rs. " + (price * count));
47 |         l8 = new Label("");
48 |         payBtn = new Button("Proceed to Payment");
49 |         backBtn = new Button("Back");
50 |
51 |         // design
52 |         setBackground(new Color(232, 240, 250));
53 |         setFont(new Font("Arial", Font.PLAIN, 15));
54 |         l1.setBackground(new Color(22, 54, 110));
55 |         l1.setForeground(Color.white);
56 |         l1.setFont(new Font("Arial", Font.BOLD, 20));
57 |         l2.setBackground(new Color(22, 54, 110));
58 |         l2.setForeground(Color.white);
59 |         l2.setFont(new Font("Arial", Font.BOLD, 15));
60 |         l7.setBackground(new Color(22, 54, 110));
61 |         l7.setForeground(Color.white);
62 |         l7.setFont(new Font("Arial", Font.BOLD, 15));
63 |         l8.setBackground(new Color(22, 54, 110));
64 |         l3.setFont(new Font("Arial", Font.BOLD, 15));
65 |         l4.setFont(new Font("Arial", Font.BOLD, 15));
66 |         l5.setFont(new Font("Arial", Font.BOLD, 15));
67 |         l6.setFont(new Font("Arial", Font.BOLD, 15));
68 |
69 |         // event handling
70 |         payBtn.addActionListener(this);
71 |         backBtn.addActionListener(this);
```

```
 72 |
 73 |         // adding components (title row and heading row)
 74 |         add(l1);
 75 |         add(l2);
 76 |         add(l7);
 77 |         add(l8);
 78 |         add(l3);
 79 |         add(l4);
 80 |         add(l5);
 81 |         add(l6);
 82 |
 83 |         // one row (seat, name, age, gender) for every passenger
 84 |         for (int i = 0; i < count; i++)
 85 |         {
 86 |             Choice gender = new Choice();
 87 |             gender.add("Male");
 88 |             gender.add("Female");
 89 |             gender.add("Other");
 90 |             add(new Label("    " + seatNos[i]));
 91 |             add(new TextField(18));
 92 |             add(new TextField(4));
 93 |             add(gender);
 94 |         }
 95 |         add(payBtn);
 96 |         add(backBtn);
 97 |
 98 |         // closing the window
 99 |         addWindowListener(this);
100 |
101 |         // creating the frame
102 |         setTitle("Passenger Details");
103 |         setSize(760, 160 + count * 55);
104 |         setLocationRelativeTo(null);
105 |         setVisible(true);
106 |     }
107 |
108 |     // event handling method
109 |     public void actionPerformed(ActionEvent e)
110 |     {
111 |         Button btn = (Button) e.getSource();
112 |         if (btn == backBtn)
113 |         {
114 |             dispose();
115 |             return;
116 |         }
117 |
118 |         // reading every passenger row: first 8 components are title and headings, then 4 for every
    > passenger
119 |         String seats = "";
120 |         for (int i = 0; i < count; i++)
121 |         {
122 |             TextField nameTf = (TextField) getComponent(8 + i * 4 + 1);
123 |             TextField ageTf = (TextField) getComponent(8 + i * 4 + 2);
124 |             Choice gender = (Choice) getComponent(8 + i * 4 + 3);
125 |             try
126 |             {
127 |                 age[i] = Integer.parseInt(ageTf.getText().trim());
128 |             }
129 |             catch (Exception ex)
130 |             {
131 |                 age[i] = 0;
132 |             }
133 |
134 |             if (nameTf.getText().trim().isEmpty() || age[i] < 1 || age[i] > 120)
135 |             {
136 |                 new MessageBox(this, "Please enter name and valid age for seat " + seatNos[i]);
137 |                 return;
138 |             }
139 |             names[i] = nameTf.getText().trim();
140 |             genders[i] = gender.getSelectedItem();
141 |             seats = seats + seatNos[i] + " ";
142 |         }
143 |
144 |         // opens the QR payment window, seats are booked only after payment is done
```

```
145 |         new PaymentFrame(this, customerId, flightId, seats.trim(), count, price * count);
146 |     }
147 |
148 |     // called by PaymentFrame after payment is done
149 |     // books all seats in one transaction and returns the PNR (throws exception if booking fails)
150 |     public String confirmBooking() throws Exception
151 |     {
152 |         Connection con = DBConnection.getConnection();
153 |         try
154 |         {
155 |             con.setAutoCommit(false); // start transaction
156 |
157 |             // 1. lock the flight row and check it
158 |             PreparedStatement ps1 = con.prepareStatement("SELECT price FROM flights WHERE flight_id =
    > ? AND departure_time > NOW() FOR UPDATE");
159 |             ps1.setInt(1, flightId);
160 |             ResultSet rs = ps1.executeQuery();
161 |             if (!rs.next())
162 |                 throw new Exception("This flight has already departed");
163 |             double ticketPrice = rs.getDouble("price");
164 |
165 |             // 2. mark every seat BOOKED only if it is still AVAILABLE
166 |             PreparedStatement ps2 = con.prepareStatement("UPDATE seats SET status = 'BOOKED' WHERE
    > seat_id = ? AND status = 'AVAILABLE'");
167 |             for (int i = 0; i < count; i++)
168 |             {
169 |                 ps2.setInt(1, seatIds[i]);
170 |                 if (ps2.executeUpdate() != 1)
171 |                     throw new Exception("Sorry, seat " + seatNos[i] + " was just booked by someone
    > else");
172 |             }
173 |
174 |             // 3. decrease available seats by number of passengers
175 |             PreparedStatement ps3 = con.prepareStatement("UPDATE flights SET available_seats =
    > available_seats - ? WHERE flight_id = ?");
176 |             ps3.setInt(1, count);
177 |             ps3.setInt(2, flightId);
178 |             ps3.executeUpdate();
179 |
180 |             // 4. make a PNR that is not used before
181 |             String pnr = TicketService.randomCode(6);
182 |             PreparedStatement ps4 = con.prepareStatement("SELECT pnr FROM bookings WHERE pnr = ?");
183 |             ps4.setString(1, pnr);
184 |             while (ps4.executeQuery().next())
185 |             {
186 |                 pnr = TicketService.randomCode(6);
187 |                 ps4.setString(1, pnr);
188 |             }
189 |
190 |             // 5. insert one row for every passenger with the same PNR
191 |             PreparedStatement ps5 = con.prepareStatement("INSERT INTO bookings (pnr, customer_id,
    > flight_id, seat_id, passenger_name, "
192 |                     + "passenger_age, passenger_gender, status, ticket_price) VALUES (?, ?, ?, ?, ?,
    > ?, ?, 'CONFIRMED', ?)");
193 |             for (int i = 0; i < count; i++)
194 |             {
195 |                 ps5.setString(1, pnr);
196 |                 ps5.setInt(2, customerId);
197 |                 ps5.setInt(3, flightId);
198 |                 ps5.setInt(4, seatIds[i]);
199 |                 ps5.setString(5, names[i]);
200 |                 ps5.setInt(6, age[i]);
201 |                 ps5.setString(7, genders[i]);
202 |                 ps5.setDouble(8, ticketPrice);
203 |                 ps5.executeUpdate();
204 |             }
205 |
206 |             // 6. save everything
207 |             con.commit();
208 |             con.close();
209 |             return pnr;
210 |         }
211 |         catch (Exception e)
212 |         {
```

```
213 |             // undo all the steps, so no seat is half booked
214 |             con.rollback();
215 |             con.close();
216 |             throw e;
217 |         }
218 |     }
219 |
220 |     // called by PaymentFrame when everything is finished, closes the booking windows
221 |     public void bookingDone()
222 |     {
223 |         if (flightsFrame != null)
224 |             flightsFrame.loadFlights("All", "All");
225 |         seatFrame.dispose();
226 |         dispose();
227 |     }
228 |
229 |     // window events (only closing is used, WindowListener needs all 7 methods)
230 |     public void windowClosing(WindowEvent we)
231 |     {
232 |         dispose();
233 |     }
234 |     public void windowOpened(WindowEvent we) {}
235 |     public void windowClosed(WindowEvent we) {}
236 |     public void windowIconified(WindowEvent we) {}
237 |     public void windowDeiconified(WindowEvent we) {}
238 |     public void windowActivated(WindowEvent we) {}
239 |     public void windowDeactivated(WindowEvent we) {}
240 | }
```

src/PaymentFrame.java

```
  1 | import java.awt.*;
  2 | import java.awt.event.*;
  3 | import java.sql.*;
  4 | import com.google.zxing.*;
  5 | import com.google.zxing.common.*;
  6 | import com.google.zxing.qrcode.*;
  7 |
  8 | // shows the total fare and a QR code, customer scans it with the phone and pays on the demo website
  9 | // seats are booked only after "Payment Done"
 10 | public class PaymentFrame extends Frame implements ActionListener, WindowListener
 11 | {
 12 |     Label l1, l3, l4, l5, l7, statusLabel;
 13 |     Button retryBtn, cancelBtn;
 14 |     Panel p1, p2;
 15 |     Canvas qrCanvas;
 16 |     BitMatrix qr = null; // black and white squares of the QR code (made by ZXing)
 17 |     PassengerDetailsFrame detailsFrame;
 18 |     int customerId, flightId, count;
 19 |     String seats, ref = "", pnr = "";
 20 |     double amount;
 21 |     boolean done = false; // true after the booking is confirmed
 22 |
 23 |     public PaymentFrame(PassengerDetailsFrame pd, int cid, int fid, String s, int c, double a)
 24 |     {
 25 |         detailsFrame = pd;
 26 |         customerId = cid;
 27 |         flightId = fid;
 28 |         seats = s;
 29 |         count = c;
 30 |         amount = a;
 31 |
 32 |         // setting the layout
 33 |         setLayout(new GridLayout(1, 2, 10, 0)); // left side details, right side QR code
 34 |
 35 |         // creating objects
 36 |         l1 = new Label("QR PAYMENT", Label.CENTER);
 37 |         l3 = new Label("");
 38 |         l4 = new Label("Seats: " + seats + "   (" + count + " passengers)");
 39 |         l5 = new Label("Total Fare: Rs. " + String.format("%.2f", amount));
 40 |         l7 = new Label("Scan the QR code with your phone");
 41 |         statusLabel = new Label("");
```

```
 42 |         retryBtn = new Button("Retry Payment");
 43 |         cancelBtn = new Button("Cancel");
 44 |         p1 = new Panel(new GridLayout(7, 1, 0, 5));
 45 |         p2 = new Panel(new GridLayout(1, 2, 10, 0));
 46 |
 47 |         // canvas that draws the QR code square by square
 48 |         qrCanvas = new Canvas()
 49 |         {
 50 |             public void paint(Graphics g)
 51 |             {
 52 |                 if (qr == null)
 53 |                     return;
 54 |                 int box = (Math.min(getWidth(), getHeight()) - 20) / qr.getWidth(); // size of one
    > small square
 55 |                 int x0 = (getWidth() - box * qr.getWidth()) / 2;
 56 |                 int y0 = (getHeight() - box * qr.getHeight()) / 2;
 57 |                 g.setColor(Color.white);
 58 |                 g.fillRect(x0, y0, box * qr.getWidth(), box * qr.getHeight());
 59 |                 g.setColor(Color.black);
 60 |                 for (int x = 0; x < qr.getWidth(); x++)
 61 |                     for (int y = 0; y < qr.getHeight(); y++)
 62 |                         if (qr.get(x, y))
 63 |                             g.fillRect(x0 + x * box, y0 + y * box, box, box);
 64 |             }
 65 |         };
 66 |
 67 |         // design
 68 |         setBackground(new Color(232, 240, 250));
 69 |         setFont(new Font("Arial", Font.PLAIN, 15));
 70 |         l1.setBackground(new Color(22, 54, 110));
 71 |         l1.setForeground(Color.white);
 72 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
 73 |         l5.setFont(new Font("Arial", Font.BOLD, 20));
 74 |         statusLabel.setFont(new Font("Arial", Font.BOLD, 14));
 75 |
 76 |         // event handling
 77 |         retryBtn.addActionListener(this);
 78 |         cancelBtn.addActionListener(this);
 79 |
 80 |         // adding components
 81 |         p2.add(retryBtn);
 82 |         p2.add(cancelBtn);
 83 |         p1.add(l1);
 84 |         p1.add(l3);
 85 |         p1.add(l4);
 86 |         p1.add(l5);
 87 |         p1.add(l7);
 88 |         p1.add(statusLabel);
 89 |         p1.add(p2);
 90 |         add(p1);
 91 |         add(qrCanvas);
 92 |
 93 |         // closing the window is same as Cancel
 94 |         addWindowListener(this);
 95 |
 96 |         // creating the frame
 97 |         setTitle("Payment");
 98 |         setSize(900, 450);
 99 |         setLocationRelativeTo(null);
100 |         setVisible(true);
101 |
102 |         newPayment();
103 |     }
104 |
105 |     // saves a new PENDING payment in MySQL and shows its QR code
106 |     void newPayment()
107 |     {
108 |         try
109 |         {
110 |             ref = "PAY" + TicketService.randomCode(6); // payment reference like PAY4K7M2X
111 |             Connection con = DBConnection.getConnection();
112 |             PreparedStatement ps = con.prepareStatement("INSERT INTO payments (payment_ref,
    > customer_id, flight_id, seats, passengers, amount, status) VALUES (?, ?, ?, ?, ?, ?, 'PENDING')");
113 |             ps.setString(1, ref);
```

```
114 |             ps.setInt(2, customerId);
115 |             ps.setInt(3, flightId);
116 |             ps.setString(4, seats);
117 |             ps.setInt(5, count);
118 |             ps.setDouble(6, amount);
119 |             ps.executeUpdate();
120 |             con.close();
121 |
122 |             // the link of the payment website is put inside the QR code
123 |             String url = "http://" + PaymentServer.getLaptopIp() + ":5173/?ref=" + ref;
124 |             qr = new QRCodeWriter().encode(url, BarcodeFormat.QR_CODE, 1, 1);
125 |             qrCanvas.repaint();
126 |             PaymentServer.current = this; // so the server can tell this window when payment is done
127 |             l3.setText("Payment Ref: " + ref);
128 |             statusLabel.setForeground(new Color(200, 120, 0));
129 |             statusLabel.setText("Waiting for payment on phone...");
130 |             retryBtn.setEnabled(false);
131 |         }
132 |         catch (Exception e)
133 |         {
134 |             System.out.println(e);
135 |             new MessageBox(this, "Error: " + e.getMessage());
136 |         }
137 |     }
138 |
139 |     // called by PaymentServer when "Payment Done" is clicked: books the seats and returns the PNR
140 |     public String paymentSuccess() throws Exception
141 |     {
142 |         try
143 |         {
144 |             pnr = detailsFrame.confirmBooking();
145 |         }
146 |         catch (Exception e)
147 |         {
148 |             // payment was done but seats are gone, so payment is marked FAILED (demo, no money taken)
149 |             PaymentServer.update("UPDATE payments SET status = 'FAILED' WHERE payment_ref = ?", ref);
150 |             statusLabel.setForeground(Color.red);
151 |             statusLabel.setText("Booking failed: " + e.getMessage());
152 |             detailsFrame.seatFrame.loadSeats();
153 |             throw e;
154 |         }
155 |
156 |         // saving the PNR with the payment
157 |         Connection con = DBConnection.getConnection();
158 |         PreparedStatement ps = con.prepareStatement("UPDATE payments SET pnr = ? WHERE payment_ref =
    > ?");
159 |         ps.setString(1, pnr);
160 |         ps.setString(2, ref);
161 |         ps.executeUpdate();
162 |         con.close();
163 |
164 |         done = true;
165 |         statusLabel.setForeground(new Color(30, 130, 70));
166 |         statusLabel.setText("PAYMENT SUCCESSFUL!  PNR: " + pnr);
167 |         return pnr;
168 |     }
169 |
170 |     // called by PaymentServer after replying to the phone, makes the PDF ticket and emails it
171 |     public void makeTicket()
172 |     {
173 |         String msg = "Confirmed! PNR: " + pnr;
174 |         try
175 |         {
176 |             String path = TicketService.makePdf(pnr);
177 |             msg = msg + " | Ticket saved";
178 |             TicketService.sendEmail(pnr, path);
179 |             msg = msg + " and emailed";
180 |         }
181 |         catch (Exception e)
182 |         {
183 |             msg = msg + " | Email failed";
184 |         }
185 |         statusLabel.setText(msg);
186 |         cancelBtn.setLabel("  Close  ");
```

```
187 |         PaymentServer.current = null;
188 |     }
189 |
190 |     // called by PaymentServer when "Payment Failed" is clicked on the phone
191 |     public void paymentFailed()
192 |     {
193 |         statusLabel.setForeground(Color.red);
194 |         statusLabel.setText("PAYMENT FAILED. Click Retry Payment.");
195 |         retryBtn.setEnabled(true);
196 |     }
197 |
198 |     // event handling method
199 |     public void actionPerformed(ActionEvent e)
200 |     {
201 |         Button btn = (Button) e.getSource();
202 |
203 |         if (btn == retryBtn)
204 |             newPayment(); // old payment stays FAILED, a new payment with a new QR code is made
205 |         else
206 |             closeWindow();
207 |     }
208 |
209 |     void closeWindow()
210 |     {
211 |         if (done)
212 |             detailsFrame.bookingDone(); // closes passenger and seat windows also
213 |         else
214 |         {
215 |             // payment not done, so it is marked FAILED and no seat is booked
216 |             PaymentServer.update("UPDATE payments SET status = 'FAILED' WHERE payment_ref = ? AND
    > status = 'PENDING'", ref);
217 |             PaymentServer.current = null;
218 |         }
219 |         dispose();
220 |     }
221 |
222 |     // window events (only closing is used, WindowListener needs all 7 methods)
223 |     public void windowClosing(WindowEvent we)
224 |     {
225 |         closeWindow();
226 |     }
227 |     public void windowOpened(WindowEvent we) {}
228 |     public void windowClosed(WindowEvent we) {}
229 |     public void windowIconified(WindowEvent we) {}
230 |     public void windowDeiconified(WindowEvent we) {}
231 |     public void windowActivated(WindowEvent we) {}
232 |     public void windowDeactivated(WindowEvent we) {}
233 | }
```

src/PaymentServer.java

```
 1 | import java.io.*;
 2 | import java.net.*;
 3 | import java.sql.*;
 4 | import com.sun.net.httpserver.*;
 5 |
 6 | // small HTTP backend (port 8085) between the demo payment website and this Java app
 7 | // it uses HttpServer which comes with Java, no extra jar is needed
 8 | //   GET  /api/payment/details?ref=PAYXXXXXX  -> payment and flight details
 9 | //   POST /api/payment/success?ref=...        -> "Payment Done" button
10 | //   POST /api/payment/fail?ref=...           -> "Payment Failed" button
11 | public class PaymentServer
12 | {
13 |     static PaymentFrame current = null; // payment window that is open on the laptop (only one at a
   > time)
14 |
15 |     public static void start()
16 |     {
17 |         try
18 |         {
19 |             HttpServer server = HttpServer.create(new InetSocketAddress(8085), 0);
20 |             server.createContext("/api/payment", new HttpHandler()
21 |             {
22 |                 public void handle(HttpExchange ex) throws IOException
23 |                 {
24 |                     handleRequest(ex);
25 |                 }
26 |             });
27 |             server.start();
28 |             System.out.println("Payment server started: http://" + getLaptopIp() + ":8085");
29 |         }
30 |         catch (Exception e)
31 |         {
32 |             System.out.println("Payment server could not start: " + e);
33 |         }
34 |     }
35 |
36 |     static void handleRequest(HttpExchange ex) throws IOException
37 |     {
38 |         String path = ex.getRequestURI().getPath();   // eg. /api/payment/success
39 |         String query = ex.getRequestURI().getQuery(); // eg. ref=PAY4K7M2X
40 |         String ref = "";
41 |         if (query != null && query.startsWith("ref="))
42 |             ref = query.substring(4).toUpperCase().replaceAll("[^A-Z0-9]", "");
43 |
44 |         // the payment window of this ref (null if it is closed)
45 |         PaymentFrame pf = null;
46 |         if (current != null && current.ref.equals(ref))
47 |             pf = current;
48 |
49 |         String reply = "{}";
50 |         if (path.endsWith("/details"))
51 |             reply = getDetails(ref);
52 |         else if (path.endsWith("/success"))
53 |             reply = paymentDone(ref, pf);
54 |         else if (path.endsWith("/fail") && update("UPDATE payments SET status = 'FAILED' WHERE
   > payment_ref = ? AND status = 'PENDING'", ref) == 1)
55 |         {
56 |             if (pf != null)
57 |                 pf.paymentFailed();
58 |             reply = message(true, "Payment failed. No seat was booked.");
59 |         }
60 |         else if (path.endsWith("/fail"))
61 |             reply = message(false, "This payment is already completed or failed.");
62 |
63 |         // sending the reply to the phone
64 |         byte[] data = reply.getBytes("UTF-8");
65 |         ex.getResponseHeaders().add("Content-Type", "application/json");
66 |         ex.getResponseHeaders().add("Access-Control-Allow-Origin", "*"); // website runs on another
   > port
67 |         ex.sendResponseHeaders(200, data.length);
68 |         OutputStream os = ex.getResponseBody();
69 |         os.write(data);
```

```
 70 |         os.close();
 71 |
 72 |         // making the PDF and email takes a few seconds, so it is done after replying to the phone
 73 |         if (path.endsWith("/success") && pf != null && pf.done)
 74 |             pf.makeTicket();
 75 |     }
 76 |
 77 |     // payment, flight and customer details for the website (as JSON text)
 78 |     static String getDetails(String ref)
 79 |     {
 80 |         try
 81 |         {
 82 |             Connection con = DBConnection.getConnection();
 83 |             PreparedStatement ps = con.prepareStatement("SELECT p.*, f.flight_no, f.airline, f.source,
    >  f.destination, f.departure_time, c.name "
 84 |                     + "FROM payments p JOIN flights f ON p.flight_id = f.flight_id JOIN customers c ON
    >  p.customer_id = c.customer_id WHERE p.payment_ref = ?");
 85 |             ps.setString(1, ref);
 86 |             ResultSet rs = ps.executeQuery();
 87 |             String json = message(false, "Payment not found. Please scan the QR code again.");
 88 |             if (rs.next())
 89 |                 json = "{\"ok\":true,\"ref\":\"" + ref + "\",\"status\":\"" + rs.getString("status") +
    >  "\",\"amount\":" + rs.getDouble("amount")
 90 |                         + ",\"seats\":\"" + rs.getString("seats") + "\",\"passengers\":" +
    > rs.getInt("passengers")
 91 |                         + ",\"flightNo\":\"" + esc(rs.getString("flight_no")) + "\",\"airline\":\"" +
    > esc(rs.getString("airline"))
 92 |                         + "\",\"from\":\"" + esc(rs.getString("source")) + "\",\"to\":\"" +
    > esc(rs.getString("destination"))
 93 |                         + "\",\"departure\":\"" + rs.getString("departure_time").substring(0, 16) +
    > "\",\"customer\":\"" + esc(rs.getString("name"))
 94 |                         + "\",\"pnr\":\"" + esc(rs.getString("pnr")) + "\"}";
 95 |             con.close();
 96 |             return json;
 97 |         }
 98 |         catch (Exception e)
 99 |         {
100 |             System.out.println(e);
101 |             return message(false, "Server error: " + e.getMessage());
102 |         }
103 |     }
104 |
105 |     // "Payment Done" clicked on phone
106 |     static String paymentDone(String ref, PaymentFrame pf)
107 |     {
108 |         // payment window is closed on the laptop
109 |         if (pf == null)
110 |         {
111 |             update("UPDATE payments SET status = 'FAILED' WHERE payment_ref = ? AND status =
    > 'PENDING'", ref);
112 |             return message(false, "This payment is closed or already completed. Please book again.");
113 |         }
114 |
115 |         // status changes only if it is still PENDING, so clicking two times cannot book two times
116 |         if (update("UPDATE payments SET status = 'SUCCESS' WHERE payment_ref = ? AND status =
    > 'PENDING'", ref) != 1)
117 |             return message(false, "This payment is already completed or failed.");
118 |
119 |         try
120 |         {
121 |             return "{\"ok\":true,\"message\":\"Payment successful. Booking confirmed!\",\"pnr\":\"" +
    > pf.paymentSuccess() + "\"}";
122 |         }
123 |         catch (Exception e)
124 |         {
125 |             return message(false, "Booking failed: " + e.getMessage() + ". No money was taken
    > (demo).");
126 |         }
127 |     }
128 |
129 |     // runs an update query for one payment and returns number of rows changed
130 |     static int update(String sql, String ref)
131 |     {
132 |         try
```

```
133 |         {
134 |             Connection con = DBConnection.getConnection();
135 |             PreparedStatement ps = con.prepareStatement(sql);
136 |             ps.setString(1, ref);
137 |             int rows = ps.executeUpdate();
138 |             con.close();
139 |             return rows;
140 |         }
141 |         catch (Exception e)
142 |         {
143 |             System.out.println(e);
144 |             return 0;
145 |         }
146 |     }
147 |
148 |     // finds the Wi-Fi IP of the laptop (eg. 192.168.1.104) so the phone can open the website
149 |     static String getLaptopIp()
150 |     {
151 |         try
152 |         {
153 |             // connecting a UDP socket does not send anything, it only tells which IP the laptop uses
    > on the network
154 |             DatagramSocket ds = new DatagramSocket();
155 |             ds.connect(InetAddress.getByName("8.8.8.8"), 80);
156 |             String ip = ds.getLocalAddress().getHostAddress();
157 |             ds.close();
158 |             return ip;
159 |         }
160 |         catch (Exception e)
161 |         {
162 |             System.out.println(e);
163 |         }
164 |         return "localhost";
165 |     }
166 |
167 |     static String message(boolean ok, String msg)
168 |     {
169 |         return "{\"ok\":" + ok + ",\"message\":\"" + esc(msg) + "\"}";
170 |     }
171 |
172 |     // makes text safe to put inside JSON
173 |     static String esc(String s)
174 |     {
175 |         if (s == null)
176 |             return "";
177 |         return s.replace("\\", "\\\\").replace("\"", "\\\"");
178 |     }
179 | }
```

src/TicketService.java

```
 1 | import java.io.*;
 2 | import java.sql.*;
 3 | import java.util.*;
 4 | import jakarta.mail.*;
 5 | import jakarta.mail.internet.*;
 6 | import org.apache.pdfbox.pdmodel.*;
 7 | import org.apache.pdfbox.pdmodel.font.*;
 8 |
 9 | // ticket details, PDF ticket, email and random codes (PNR / payment reference)
10 | public class TicketService
11 | {
12 |     // booking details as text (used in TextArea, PDF and email), one row of the query = one passenger
13 |     public static String getDetails(String pnr) throws Exception
14 |     {
15 |         Connection con = DBConnection.getConnection();
16 |         PreparedStatement ps = con.prepareStatement("SELECT b.*, s.seat_no, c.name, c.email, f.* FROM
   > bookings b "
17 |                 + "JOIN seats s ON b.seat_id = s.seat_id JOIN customers c ON b.customer_id =
   > c.customer_id "
18 |                 + "JOIN flights f ON b.flight_id = f.flight_id WHERE b.pnr = ? ORDER BY s.seat_id");
19 |         ps.setString(1, pnr);
20 |         ResultSet rs = ps.executeQuery();
21 |
22 |         String info = "", list = "";
23 |         int n = 0;
24 |         double total = 0;
25 |         while (rs.next())
26 |         {
27 |             // flight details are same for all passengers, so they are taken from the first row
28 |             if (n == 0)
29 |                 info = "PNR            : " + pnr + "\nBooked By      : " + rs.getString("name") + " ("
   >  + rs.getString("email") + ")"
30 |                         + "\nFlight         : " + rs.getString("flight_no") + " (" +
   > rs.getString("airline") + ")"
31 |                         + "\nRoute          : " + rs.getString("source") + " -> " +
   > rs.getString("destination")
32 |                         + "\nDeparture      : " + rs.getString("departure_time").substring(0, 16)
33 |                         + "\nArrival        : " + rs.getString("arrival_time").substring(0, 16)
34 |                         + "\nBooking Date   : " + rs.getString("booking_date").substring(0, 16) +
   > "\n";
35 |
36 |             list = list + String.format("%-20s %-12s %-6s %-10s %.2f\n",
   > rs.getString("passenger_name"),
37 |                     rs.getInt("passenger_age") + " / " + rs.getString("passenger_gender"),
   > rs.getString("seat_no"),
38 |                     rs.getString("status"), rs.getDouble("ticket_price"));
39 |             if (rs.getString("status").equals("CONFIRMED"))
40 |                 total = total + rs.getDouble("ticket_price"); // cancelled tickets are not counted
41 |             n++;
42 |         }
43 |         con.close();
44 |
45 |         if (n == 0)
46 |             throw new Exception("Booking not found");
47 |         return info + "\nPASSENGERS (" + n + ")\n" + String.format("%-20s %-12s %-6s %-10s %s\n",
   > "Name", "Age/Gender", "Seat", "Status", "Price")
48 |                 + list + String.format("\nTotal Fare     : Rs. %.2f", total);
49 |     }
50 |
51 |     // makes tickets/ticket_PNR.pdf with the booking details and returns the file path
52 |     public static String makePdf(String pnr) throws Exception
53 |     {
54 |         String[] lines = getDetails(pnr).split("\n");
55 |         new File("tickets").mkdirs();
56 |         String path = "tickets/ticket_" + pnr + ".pdf";
57 |
58 |         PDDocument doc = new PDDocument();
59 |         PDPage page = new PDPage();
60 |         doc.addPage(page);
61 |         PDPageContentStream cs = new PDPageContentStream(doc, page);
62 |
63 |         // heading
```

```
 64 |         cs.beginText();
 65 |         cs.setFont(new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD), 22);
 66 |         cs.newLineAtOffset(50, 730);
 67 |         cs.showText("SkyWay Airline Reservation System");
 68 |         cs.setFont(new PDType1Font(Standard14Fonts.FontName.HELVETICA), 14);
 69 |         cs.newLineAtOffset(0, -25);
 70 |         cs.showText("Electronic Ticket / Boarding Pass");
 71 |         cs.endText();
 72 |
 73 |         // booking details line by line (Courier font keeps the columns straight)
 74 |         cs.beginText();
 75 |         cs.setFont(new PDType1Font(Standard14Fonts.FontName.COURIER), 11);
 76 |         cs.setLeading(16);
 77 |         cs.newLineAtOffset(60, 650);
 78 |         for (int i = 0; i < lines.length; i++)
 79 |         {
 80 |             cs.showText(lines[i]);
 81 |             cs.newLine();
 82 |         }
 83 |         cs.showText("");
 84 |         cs.newLine();
 85 |         cs.showText("Please reach the airport 2 hours before departure with a valid photo ID.");
 86 |         cs.endText();
 87 |
 88 |         // box around the details
 89 |         cs.addRect(40, 650 - lines.length * 16, 530, lines.length * 16 + 30);
 90 |         cs.stroke();
 91 |         cs.close();
 92 |         doc.save(path);
 93 |         doc.close();
 94 |         return path;
 95 |     }
 96 |
 97 |     // sends the PDF ticket to the customer's email using Gmail
 98 |     public static void sendEmail(String pnr, String pdfPath) throws Exception
 99 |     {
100 |         // reading gmail id and app password from config.properties
101 |         Properties config = new Properties();
102 |         FileInputStream fis = new FileInputStream("config.properties");
103 |         config.load(fis);
104 |         fis.close();
105 |         String from = config.getProperty("mail.username", "").trim();
106 |         String pass = config.getProperty("mail.password", "").trim();
107 |         if (from.isEmpty() || pass.isEmpty())
108 |             throw new Exception("Email not set. Add your Gmail in config.properties");
109 |
110 |         // finding the customer email
111 |         Connection con = DBConnection.getConnection();
112 |         PreparedStatement ps = con.prepareStatement("SELECT c.name, c.email FROM bookings b JOIN
    > customers c ON b.customer_id = c.customer_id WHERE b.pnr = ?");
113 |         ps.setString(1, pnr);
114 |         ResultSet rs = ps.executeQuery();
115 |         rs.next();
116 |         String name = rs.getString("name");
117 |         String to = rs.getString("email");
118 |         con.close();
119 |
120 |         // gmail smtp settings
121 |         Properties props = new Properties();
122 |         props.put("mail.smtp.auth", "true");
123 |         props.put("mail.smtp.starttls.enable", "true");
124 |         props.put("mail.smtp.host", "smtp.gmail.com");
125 |         props.put("mail.smtp.port", "587");
126 |         Session session = Session.getInstance(props, new Authenticator()
127 |         {
128 |             protected PasswordAuthentication getPasswordAuthentication()
129 |             {
130 |                 return new PasswordAuthentication(from, pass);
131 |             }
132 |         });
133 |
134 |         // mail with text and pdf attachment
135 |         MimeMessage msg = new MimeMessage(session);
136 |         msg.setFrom(new InternetAddress(from));
```

```
137 |         msg.setRecipients(Message.RecipientType.TO, InternetAddress.parse(to));
138 |         msg.setSubject("Your E-Ticket - PNR " + pnr);
139 |         MimeBodyPart textPart = new MimeBodyPart();
140 |         textPart.setText("Dear " + name + ",\n\nThank you for booking with us. Your ticket is
    > attached.\n\n" + getDetails(pnr) + "\n\nHappy journey!\nSkyWay Airlines");
141 |         MimeBodyPart filePart = new MimeBodyPart();
142 |         filePart.attachFile(new File(pdfPath));
143 |         Multipart mp = new MimeMultipart();
144 |         mp.addBodyPart(textPart);
145 |         mp.addBodyPart(filePart);
146 |         msg.setContent(mp);
147 |         Transport.send(msg);
148 |     }
149 |
150 |     // random code like K7MX2P (used for PNR and payment reference)
151 |     public static String randomCode(int length)
152 |     {
153 |         String chars = "ACDEFHJKLMNPQRTUVWXY23456789"; // no B G I O S Z, they look like 8 6 1 0 5 2
154 |         Random r = new Random();
155 |         String code = "";
156 |         for (int i = 0; i < length; i++)
157 |             code = code + chars.charAt(r.nextInt(chars.length()));
158 |         return code;
159 |     }
160 | }
```

src/MyBookingsFrame.java

```
  1 | import java.awt.*;
  2 | import java.awt.event.*;
  3 | import java.sql.*;
  4 |
  5 | // customer can see his bookings, search by PNR, cancel and get the ticket again
  6 | public class MyBookingsFrame extends Frame implements ActionListener, ItemListener, WindowListener
  7 | {
  8 |     Label l1, l2, l3;
  9 |     TextField pnrTf;
 10 |     List bookingList;
 11 |     TextArea detailsArea;
 12 |     Button searchBtn, showAllBtn, cancelBtn, ticketBtn, backBtn;
 13 |     Panel p1, p2, p3;
 14 |     int customerId;
 15 |     String[] pnrs = new String[500];   // pnr of every row in the list
 16 |     int[] bookingIds = new int[500];   // booking_id of every row (one row = one passenger)
 17 |     String[] names = new String[500];  // passenger name of every row
 18 |     int total = 0;                     // number of rows in the list
 19 |
 20 |     public MyBookingsFrame(int id)
 21 |     {
 22 |         customerId = id;
 23 |
 24 |         // setting the layout (no layout manager, every component gets x, y, width, height)
 25 |         setLayout(new GridLayout(3, 1, 0, 5)); // top part, list of bookings, details of the selected
    > booking
 26 |
 27 |         // creating objects
 28 |         l1 = new Label("MY BOOKINGS", Label.CENTER);
 29 |         l2 = new Label("PNR", Label.RIGHT);
 30 |         l3 = new Label(String.format(" %-7s %-7s %-22s %-17s %-5s %-18s %s", "PNR", "Flight", "Route",
    >  "Departure", "Seat", "Passenger", "Status"));
 31 |         pnrTf = new TextField(10);
 32 |         bookingList = new List(8);
 33 |         detailsArea = new TextArea(14, 60);
 34 |         searchBtn = new Button("Search");
 35 |         showAllBtn = new Button("List All");
 36 |         cancelBtn = new Button("Cancel Ticket");
 37 |         ticketBtn = new Button("Get Ticket (PDF + Email)");
 38 |         backBtn = new Button("Back");
 39 |
 40 |         p1 = new Panel(new GridLayout(4, 1, 5, 8));
 41 |         p2 = new Panel(new GridLayout(1, 4, 10, 0));
 42 |         p3 = new Panel(new GridLayout(1, 3, 10, 0));
```

```
 43 |         // design
 44 |         setBackground(new Color(232, 240, 250));
 45 |         setFont(new Font("Arial", Font.PLAIN, 15));
 46 |         l1.setBackground(new Color(22, 54, 110));
 47 |         l1.setForeground(Color.white);
 48 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
 49 |         l3.setFont(new Font("Monospaced", Font.BOLD, 14));
 50 |         bookingList.setFont(new Font("Monospaced", Font.PLAIN, 14));
 51 |         detailsArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
 52 |         detailsArea.setEditable(false);
 53 |
 54 |         // event handling
 55 |         searchBtn.addActionListener(this);
 56 |         showAllBtn.addActionListener(this);
 57 |         cancelBtn.addActionListener(this);
 58 |         ticketBtn.addActionListener(this);
 59 |         backBtn.addActionListener(this);
 60 |         bookingList.addItemListener(this);
 61 |
 62 |         // adding components
 63 |         p2.add(l2);
 64 |         p2.add(pnrTf);
 65 |         p2.add(searchBtn);
 66 |         p2.add(showAllBtn);
 67 |         p3.add(cancelBtn);
 68 |         p3.add(ticketBtn);
 69 |         p3.add(backBtn);
 70 |         p1.add(l1);
 71 |         p1.add(p2);
 72 |         p1.add(p3);
 73 |         p1.add(l3);
 74 |         add(p1);
 75 |         add(bookingList);
 76 |         add(detailsArea);
 77 |
 78 |         // closing the window
 79 |         addWindowListener(this);
 80 |
 81 |         // creating the frame
 82 |         setTitle("My Bookings");
 83 |         setSize(1000, 740);
 84 |         setLocationRelativeTo(null);
 85 |         setVisible(true);
 86 |
 87 |         loadBookings("");
 88 |     }
 89 |
 90 |     // shows bookings of this customer, if pnr is empty it shows all of them
 91 |     void loadBookings(String pnr)
 92 |     {
 93 |         bookingList.removeAll();
 94 |         total = 0;
 95 |         try
 96 |         {
 97 |             Connection con = DBConnection.getConnection();
 98 |             PreparedStatement ps = con.prepareStatement("SELECT b.*, s.seat_no, f.flight_no, f.source,
    >  f.destination, f.departure_time FROM bookings b "
 99 |                     + "JOIN seats s ON b.seat_id = s.seat_id JOIN flights f ON b.flight_id =
    > f.flight_id "
100 |                     + "WHERE b.customer_id = ? AND (? = '' OR b.pnr = ?) ORDER BY b.booking_date DESC,
    >  b.pnr, s.seat_id");
101 |             ps.setInt(1, customerId);
102 |             ps.setString(2, pnr);
103 |             ps.setString(3, pnr);
104 |             ResultSet rs = ps.executeQuery();
105 |             while (rs.next() && total < 500)
106 |             {
107 |                 pnrs[total] = rs.getString("pnr");
108 |                 bookingIds[total] = rs.getInt("booking_id");
109 |                 names[total] = rs.getString("passenger_name");
110 |                 total++;
111 |                 bookingList.add(String.format("%-7s %-7s %-22s %-17s %-5s %-18s %s",
    > rs.getString("pnr"), rs.getString("flight_no"),
112 |                         rs.getString("source") + " -> " + rs.getString("destination"),
```

```
    > rs.getString("departure_time").substring(0, 16),
113 |                         rs.getString("seat_no"), rs.getString("passenger_name"),
    > rs.getString("status")));
114 |             }
115 |             con.close();
116 |             detailsArea.setText("Total tickets: " + total + "\nClick on a booking to see the
    > details.");
117 |         }
118 |         catch (Exception e)
119 |         {
120 |             System.out.println(e);
121 |             new MessageBox(this, "Error: " + e.getMessage());
122 |         }
123 |     }
124 |
125 |     // event handling method
126 |     public void actionPerformed(ActionEvent e)
127 |     {
128 |         Button btn = (Button) e.getSource();
129 |         int i = bookingList.getSelectedIndex();
130 |
131 |         if (btn == backBtn)
132 |             dispose();
133 |         else if (btn == searchBtn)
134 |             loadBookings(pnrTf.getText().trim().toUpperCase());
135 |         else if (btn == showAllBtn)
136 |         {
137 |             pnrTf.setText("");
138 |             loadBookings("");
139 |         }
140 |         else if (i < 0)
141 |             new MessageBox(this, "Please select a booking from the list first");
142 |         else if (btn == ticketBtn)
143 |         {
144 |             // makes the PDF again and emails it
145 |             String msg = "";
146 |             try
147 |             {
148 |                 String path = TicketService.makePdf(pnrs[i]);
149 |                 msg = "Ticket saved at: " + path;
150 |                 TicketService.sendEmail(pnrs[i], path);
151 |                 msg = msg + "\nTicket sent to your email";
152 |             }
153 |             catch (Exception ex)
154 |             {
155 |                 msg = msg + "\nEmail failed: " + ex.getMessage();
156 |             }
157 |             new MessageBox(this, msg);
158 |         }
159 |         else if (new MessageBox(this, "Cancel the ticket of " + names[i] + " (PNR " + pnrs[i] + ")?",
    > true).yes)
160 |             cancelTicket(i);
161 |     }
162 |
163 |     // cancels the ticket of one passenger in one transaction (other passengers of the PNR are not
    > cancelled)
164 |     void cancelTicket(int i)
165 |     {
166 |         Connection con = null;
167 |         try
168 |         {
169 |             con = DBConnection.getConnection();
170 |             con.setAutoCommit(false); // start transaction
171 |
172 |             // 1. lock the booking row and check it
173 |             PreparedStatement ps1 = con.prepareStatement("SELECT b.*, f.departure_time < NOW() AS
    > departed FROM bookings b "
174 |                     + "JOIN flights f ON b.flight_id = f.flight_id WHERE b.booking_id = ? AND
    > b.customer_id = ? FOR UPDATE");
175 |             ps1.setInt(1, bookingIds[i]);
176 |             ps1.setInt(2, customerId);
177 |             ResultSet rs = ps1.executeQuery();
178 |             if (!rs.next() || !rs.getString("status").equals("CONFIRMED"))
179 |                 throw new Exception("This ticket is already cancelled");
```

```
180 |             if (rs.getInt("departed") == 1)
181 |                 throw new Exception("Flight has already departed, cannot cancel");
182 |
183 |             // 2. mark booking cancelled, free the seat and increase available seats
184 |             PreparedStatement ps2 = con.prepareStatement("UPDATE bookings SET status = 'CANCELLED'
    > WHERE booking_id = ?");
185 |             ps2.setInt(1, bookingIds[i]);
186 |             ps2.executeUpdate();
187 |             PreparedStatement ps3 = con.prepareStatement("UPDATE seats SET status = 'AVAILABLE' WHERE
    > seat_id = ?");
188 |             ps3.setInt(1, rs.getInt("seat_id"));
189 |             ps3.executeUpdate();
190 |             PreparedStatement ps4 = con.prepareStatement("UPDATE flights SET available_seats =
    > available_seats + 1 WHERE flight_id = ? AND available_seats < total_seats");
191 |             ps4.setInt(1, rs.getInt("flight_id"));
192 |             ps4.executeUpdate();
193 |
194 |             // 3. save everything
195 |             con.commit();
196 |             con.close();
197 |             new MessageBox(this, "Ticket of " + names[i] + " cancelled. Seat has been released.");
198 |             loadBookings("");
199 |         }
200 |         catch (Exception e)
201 |         {
202 |             // undo all the steps
203 |             try
204 |             {
205 |                 con.rollback();
206 |                 con.close();
207 |             }
208 |             catch (Exception ex)
209 |             {
210 |                 System.out.println(ex);
211 |             }
212 |             new MessageBox(this, e.getMessage());
213 |         }
214 |     }
215 |
216 |     // when a booking is clicked in the list, show its details
217 |     public void itemStateChanged(ItemEvent e)
218 |     {
219 |         try
220 |         {
221 |             detailsArea.setText(TicketService.getDetails(pnrs[bookingList.getSelectedIndex()]));
222 |         }
223 |         catch (Exception ex)
224 |         {
225 |             detailsArea.setText("Error: " + ex.getMessage());
226 |         }
227 |     }
228 |
229 |     // window events (only closing is used, WindowListener needs all 7 methods)
230 |     public void windowClosing(WindowEvent we)
231 |     {
232 |         dispose();
233 |     }
234 |     public void windowOpened(WindowEvent we) {}
235 |     public void windowClosed(WindowEvent we) {}
236 |     public void windowIconified(WindowEvent we) {}
237 |     public void windowDeiconified(WindowEvent we) {}
238 |     public void windowActivated(WindowEvent we) {}
239 |     public void windowDeactivated(WindowEvent we) {}
240 | }
```

src/ViewBookingsFrame.java

```
 1 | import java.awt.*;
 2 | import java.awt.event.*;
 3 | import java.sql.*;
 4 |
 5 | // admin can see all bookings and search a booking by PNR
 6 | public class ViewBookingsFrame extends Frame implements ActionListener, ItemListener, WindowListener
 7 | {
 8 |     Label l1, l2, l3;
 9 |     TextField pnrTf;
10 |     List bookingList;
11 |     TextArea detailsArea;
12 |     Button searchBtn, showAllBtn, backBtn;
13 |     Panel p1, p2;
14 |     String[] pnrs = new String[1000]; // pnr of every row in the list
15 |     int total = 0;                    // number of rows in the list
16 |
17 |     public ViewBookingsFrame()
18 |     {
19 |         // setting the layout (no layout manager, every component gets x, y, width, height)
20 |         setLayout(new GridLayout(3, 1, 0, 5)); // top part, list of bookings, details of the selected
   > booking
21 |
22 |         // creating objects
23 |         l1 = new Label("ALL BOOKINGS", Label.CENTER);
24 |         l2 = new Label("PNR", Label.RIGHT);
25 |         l3 = new Label(String.format(" %-7s %-7s %-22s %-17s %-5s %-10s %s", "PNR", "Flight", "Route",
   >  "Departure", "Seat", "Status", "Passenger"));
26 |         pnrTf = new TextField(10);
27 |         bookingList = new List(10);
28 |         detailsArea = new TextArea(14, 60);
29 |         searchBtn = new Button("Search");
30 |         showAllBtn = new Button("List All");
31 |         backBtn = new Button("Back");
32 |
33 |         p1 = new Panel(new GridLayout(3, 1, 5, 8));
34 |         p2 = new Panel(new GridLayout(1, 5, 10, 0));
35 |         // design
36 |         setBackground(new Color(232, 240, 250));
37 |         setFont(new Font("Arial", Font.PLAIN, 15));
38 |         l1.setBackground(new Color(22, 54, 110));
39 |         l1.setForeground(Color.white);
40 |         l1.setFont(new Font("Arial", Font.BOLD, 22));
41 |         l3.setFont(new Font("Monospaced", Font.BOLD, 14));
42 |         bookingList.setFont(new Font("Monospaced", Font.PLAIN, 14));
43 |         detailsArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
44 |         detailsArea.setEditable(false);
45 |
46 |         // event handling
47 |         searchBtn.addActionListener(this);
48 |         showAllBtn.addActionListener(this);
49 |         backBtn.addActionListener(this);
50 |         bookingList.addItemListener(this);
51 |
52 |         // adding components
53 |         p2.add(l2);
54 |         p2.add(pnrTf);
55 |         p2.add(searchBtn);
56 |         p2.add(showAllBtn);
57 |         p2.add(backBtn);
58 |         p1.add(l1);
59 |         p1.add(p2);
60 |         p1.add(l3);
61 |         add(p1);
62 |         add(bookingList);
63 |         add(detailsArea);
64 |
65 |         // closing the window
66 |         addWindowListener(this);
67 |
68 |         // creating the frame
69 |         setTitle("All Bookings");
70 |         setSize(900, 720);
```

```
71 |         setLocationRelativeTo(null);
72 |         setVisible(true);
73 |
74 |         loadBookings("");
75 |     }
76 |
77 |     // shows bookings in the list, if pnr is empty it shows all bookings
78 |     void loadBookings(String pnr)
79 |     {
80 |         bookingList.removeAll();
81 |         total = 0;
82 |         try
83 |         {
84 |             Connection con = DBConnection.getConnection();
85 |             PreparedStatement ps = con.prepareStatement("SELECT b.*, s.seat_no, f.flight_no, f.source,
   >  f.destination, f.departure_time FROM bookings b "
86 |                     + "JOIN seats s ON b.seat_id = s.seat_id JOIN flights f ON b.flight_id =
   > f.flight_id "
87 |                     + "WHERE ? = '' OR b.pnr = ? ORDER BY b.booking_date DESC, b.pnr, s.seat_id");
88 |             ps.setString(1, pnr);
89 |             ps.setString(2, pnr);
90 |             ResultSet rs = ps.executeQuery();
91 |             while (rs.next() && total < 1000)
92 |             {
93 |                 pnrs[total] = rs.getString("pnr");
94 |                 total++;
95 |                 bookingList.add(String.format("%-7s %-7s %-22s %-17s %-5s %-10s %s",
   > rs.getString("pnr"), rs.getString("flight_no"),
96 |                         rs.getString("source") + " -> " + rs.getString("destination"),
   > rs.getString("departure_time").substring(0, 16),
97 |                         rs.getString("seat_no"), rs.getString("status"),
   > rs.getString("passenger_name")));
98 |             }
99 |             con.close();
100 |             detailsArea.setText("Total tickets: " + total + "\nClick on a booking to see the
   > details.");
101 |         }
102 |         catch (Exception e)
103 |         {
104 |             System.out.println(e);
105 |             new MessageBox(this, "Error: " + e.getMessage());
106 |         }
107 |     }
108 |
109 |     // event handling method
110 |     public void actionPerformed(ActionEvent e)
111 |     {
112 |         Button btn = (Button) e.getSource();
113 |
114 |         if (btn == backBtn)
115 |             dispose();
116 |         else if (btn == searchBtn)
117 |             loadBookings(pnrTf.getText().trim().toUpperCase());
118 |         else
119 |         {
120 |             pnrTf.setText("");
121 |             loadBookings("");
122 |         }
123 |     }
124 |
125 |     // when a booking is clicked in the list, show its details
126 |     public void itemStateChanged(ItemEvent e)
127 |     {
128 |         try
129 |         {
130 |             detailsArea.setText(TicketService.getDetails(pnrs[bookingList.getSelectedIndex()]));
131 |         }
132 |         catch (Exception ex)
133 |         {
134 |             detailsArea.setText("Error: " + ex.getMessage());
135 |         }
136 |     }
137 |
138 |     // window events (only closing is used, WindowListener needs all 7 methods)
```

```
    public void windowClosing(WindowEvent we)
    {
        dispose();
    }
    public void windowOpened(WindowEvent we) {}
    public void windowClosed(WindowEvent we) {}
    public void windowIconified(WindowEvent we) {}
    public void windowDeiconified(WindowEvent we) {}
    public void windowActivated(WindowEvent we) {}
    public void windowDeactivated(WindowEvent we) {}
}
```

database/airline_db.sql

```
-- Airline Reservation Management System
-- Import this file in phpMyAdmin (XAMPP) -> Import tab -> choose this file -> Go

CREATE DATABASE IF NOT EXISTS airline_db;
USE airline_db;

DROP TABLE IF EXISTS payments;
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS seats;
DROP TABLE IF EXISTS flights;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS admins;

CREATE TABLE admins (
    admin_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15) NOT NULL,
    password VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE flights (
    flight_id INT AUTO_INCREMENT PRIMARY KEY,
    flight_no VARCHAR(10) NOT NULL,
    airline VARCHAR(50) NOT NULL,
    source VARCHAR(50) NOT NULL,
    destination VARCHAR(50) NOT NULL,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    total_seats INT NOT NULL,
    available_seats INT NOT NULL,
    UNIQUE (flight_no, departure_time),
    INDEX idx_route (source, destination, departure_time),
    CHECK (available_seats >= 0 AND available_seats <= total_seats)
) ENGINE=InnoDB;

CREATE TABLE seats (
    seat_id INT AUTO_INCREMENT PRIMARY KEY,
    flight_id INT NOT NULL,
    seat_no VARCHAR(5) NOT NULL,
    status ENUM('AVAILABLE', 'BOOKED') NOT NULL DEFAULT 'AVAILABLE',
    UNIQUE (flight_id, seat_no),
    FOREIGN KEY (flight_id) REFERENCES flights(flight_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    pnr VARCHAR(10) NOT NULL,              -- same PNR for all passengers of one booking
    customer_id INT NOT NULL,
    flight_id INT NOT NULL,
```

```
    seat_id INT NOT NULL,
    passenger_name VARCHAR(100) NOT NULL,
    passenger_age INT NOT NULL,
    passenger_gender VARCHAR(10) NOT NULL,
    booking_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status ENUM('CONFIRMED', 'CANCELLED') NOT NULL DEFAULT 'CONFIRMED',
    ticket_price DECIMAL(10,2) NOT NULL,
    INDEX idx_customer (customer_id),
    INDEX idx_pnr (pnr),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (flight_id) REFERENCES flights(flight_id) ON DELETE CASCADE,
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- dummy QR payments (demo only, no real money)
CREATE TABLE IF NOT EXISTS payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    payment_ref VARCHAR(12) NOT NULL UNIQUE,      -- shown in QR code and website
    customer_id INT NOT NULL,
    flight_id INT NOT NULL,
    seats VARCHAR(100) NOT NULL,                  -- eg. 1A, 1B
    passengers INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status ENUM('PENDING', 'SUCCESS', 'FAILED') NOT NULL DEFAULT 'PENDING',
    pnr VARCHAR(10),                              -- filled after booking is confirmed
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (flight_id) REFERENCES flights(flight_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ---------- sample data ----------

INSERT INTO admins (username, password) VALUES ('admin', 'admin123');

INSERT INTO customers (name, email, phone, password)
VALUES ('Rahul Sharma', 'rahul@gmail.com', '9876543210', '1234');

INSERT INTO flights (flight_no, airline, source, destination, departure_time, arrival_time, price,
> total_seats, available_seats) VALUES
('AI101', 'Air India', 'Mumbai', 'Delhi', '2026-12-10 08:00:00', '2026-12-10 10:15:00', 4500.00, 12,
> 12),
('6E202', 'IndiGo', 'Delhi', 'Mumbai', '2026-12-12 18:30:00', '2026-12-12 20:40:00', 3900.00, 12, 12),
('UK303', 'Vistara', 'Mumbai', 'Bangalore', '2026-12-15 06:45:00', '2026-12-15 08:30:00', 5200.00, 12,
>  12);

-- 12 seats for each sample flight (2 rows x 6 seats)
INSERT INTO seats (flight_id, seat_no) VALUES
(1,'1A'),(1,'1B'),(1,'1C'),(1,'1D'),(1,'1E'),(1,'1F'),(1,'2A'),(1,'2B'),(1,'2C'),(1,'2D'),(1,'2E'),(1,
> '2F'),
(2,'1A'),(2,'1B'),(2,'1C'),(2,'1D'),(2,'1E'),(2,'1F'),(2,'2A'),(2,'2B'),(2,'2C'),(2,'2D'),(2,'2E'),(2,
> '2F'),
(3,'1A'),(3,'1B'),(3,'1C'),(3,'1D'),(3,'1E'),(3,'1F'),(3,'2A'),(3,'2B'),(3,'2C'),(3,'2D'),(3,'2E'),(3,
> '2F');
```

database/payments_table.sql

```
-- run this once in phpMyAdmin (SQL tab) if your database already exists
-- (airline_db.sql already has this table for a fresh import)
USE airline_db;

CREATE TABLE IF NOT EXISTS payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    payment_ref VARCHAR(12) NOT NULL UNIQUE,      -- shown in QR code and website
    customer_id INT NOT NULL,
    flight_id INT NOT NULL,
    seats VARCHAR(100) NOT NULL,                  -- eg. 1A, 1B
    passengers INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status ENUM('PENDING', 'SUCCESS', 'FAILED') NOT NULL DEFAULT 'PENDING',
    pnr VARCHAR(10),                              -- filled after booking is confirmed
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (flight_id) REFERENCES flights(flight_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

payment-website/src/App.jsx

Display-only glyphs in this listing are spelled out as [plane], Rs. and a hyphen for PDF font compatibility. The original source keeps its glyphs.

```
import { useEffect, useState } from 'react'

// Java payment server runs on the same laptop (port 8085)
const API = 'http://' + window.location.hostname + ':8085/api/payment'

function App() {
  // payment reference comes from the QR code link, eg. ?ref=PAY4K7M2X
  const ref = new URLSearchParams(window.location.search).get('ref')

  const [payment, setPayment] = useState(null)
  const [error, setError] = useState('')
  const [note, setNote] = useState('')
  const [busy, setBusy] = useState(false)

  // load payment details when the page opens
  useEffect(() => {
    loadDetails()
  }, [])

  function loadDetails() {
    if (!ref) {
      setError('No payment reference found. Please scan the QR code again.')
      return
    }
    fetch(API + '/details?ref=' + ref)
      .then((res) => res.json())
      .then((data) => {
        if (data.ok) setPayment(data)
        else setError(data.message)
      })
      .catch(() => setError('Cannot connect to the airline server. Make sure your phone and laptop are
>  on the same Wi-Fi.'))
  }

  // tells the Java server: success, fail or retry
  function send(action) {
    setBusy(true)
    fetch(API + '/' + action + '?ref=' + ref, { method: 'POST' })
      .then((res) => res.json())
      .then((data) => {
        setNote(data.message)
        setBusy(false)
        loadDetails()
      })
      .catch(() => {
        setNote('Cannot connect to the airline server.')
```

```
        setBusy(false)
      })
  }

  return (
    <div className="page">
      <header className="header">
        <div className="logo">[plane] SkyWay Airlines</div>
        <div className="sub">Flight Ticket Payment</div>
      </header>

      {error && <div className="card error">{error}</div>}

      {!error && !payment && <div className="card">Loading payment details...</div>}

      {payment && (
        <div className="card">
          <div className="ref">
            Booking Reference <b>{payment.ref}</b>
          </div>

          <div className="route">
            <div>
              <div className="city">{payment.from}</div>
              <div className="small">From</div>
            </div>
            <div className="plane">[plane]</div>
            <div>
              <div className="city">{payment.to}</div>
              <div className="small">To</div>
            </div>
          </div>

          <table className="details">
            <tbody>
              <tr><td>Flight</td><td>{payment.flightNo} ({payment.airline})</td></tr>
              <tr><td>Departure</td><td>{payment.departure}</td></tr>
              <tr><td>Passengers</td><td>{payment.passengers}</td></tr>
              <tr><td>Seats</td><td>{payment.seats}</td></tr>
              <tr><td>Booked by</td><td>{payment.customer}</td></tr>
            </tbody>
          </table>

          <div className="amount">
            <span>Total Amount</span>
            Rs. {payment.amount.toFixed(2)}
          </div>

          {payment.status === 'PENDING' && (
            <div className="buttons">
              <button className="done" disabled={busy} onClick={() => send('success')}>
                Payment Done
              </button>
              <button className="fail" disabled={busy} onClick={() => send('fail')}>
                Payment Failed
              </button>
            </div>
          )}

          {payment.status === 'SUCCESS' && (
            <div className="result success">
              <b>Payment Successful</b>
              Your booking is confirmed. PNR: <b>{payment.pnr}</b>
              <br />Your ticket is shown on the laptop and sent to your email.
            </div>
          )}

          {payment.status === 'FAILED' && (
            <div className="result failed">
              <b>Payment Failed</b>
              No seat was booked.
              <br />Click <b>Retry Payment</b> on the laptop and scan the new QR code.
            </div>
          )}
```

```
120 |
121 |           {note && <div className="note">{note}</div>}
122 |         </div>
123 |       )}
124 |
125 |       <footer>SkyWay Airline Reservation System - Student Project</footer>
126 |     </div>
127 |   )
128 | }
129 |
130 | export default App
```

payment-website/src/main.jsx

```
1 | import React from 'react'
2 | import ReactDOM from 'react-dom/client'
3 | import App from './App.jsx'
4 | import './App.css'
5 |
6 | ReactDOM.createRoot(document.getElementById('root')).render(<App />)
```

payment-website/src/App.css

```
1 | /* same colours as the Java AWT app */
2 | :root {
3 |   --navy: #16366e;
4 |   --light: #e8f0fa;
5 |   --green: #1e7d46;
6 |   --red: #c0392b;
7 | }
8 |
9 | * {
10 |   box-sizing: border-box;
11 | }
12 |
13 | body {
14 |   margin: 0;
15 |   font-family: Arial, Helvetica, sans-serif;
16 |   background: var(--light);
17 |   color: #222;
18 | }
19 |
20 | .page {
21 |   max-width: 480px;
22 |   margin: 0 auto;
23 |   padding-bottom: 30px;
24 | }
25 |
26 | .header {
27 |   background: var(--navy);
28 |   color: white;
29 |   text-align: center;
30 |   padding: 22px 10px 16px;
31 | }
32 |
33 | .logo {
34 |   font-size: 24px;
35 |   font-weight: bold;
36 | }
37 |
38 | .sub {
39 |   font-size: 14px;
40 |   opacity: 0.8;
41 |   margin-top: 4px;
42 | }
43 |
44 | .card {
45 |   background: white;
46 |   margin: 16px;
47 |   padding: 18px;
```

```
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.card.error {
  color: var(--red);
  text-align: center;
}

.ref {
  text-align: center;
  font-size: 13px;
  color: #666;
}

.ref b {
  display: block;
  font-size: 20px;
  color: var(--navy);
  letter-spacing: 2px;
  margin-top: 4px;
}

.route {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin: 18px 0;
  text-align: center;
}

.city {
  font-size: 20px;
  font-weight: bold;
  text-transform: capitalize;
}

.small {
  font-size: 12px;
  color: #888;
}

.plane {
  font-size: 26px;
  color: var(--navy);
}

.details {
  width: 100%;
  border-collapse: collapse;
  font-size: 15px;
}

.details td {
  padding: 8px 0;
  border-bottom: 1px solid #eee;
}

.details td:first-child {
  color: #777;
}

.details td:last-child {
  text-align: right;
  font-weight: bold;
}

.amount {
  text-align: center;
  font-size: 30px;
  font-weight: bold;
  color: var(--navy);
  margin: 20px 0 10px;
}
```

```
122 |
123 | .amount span {
124 |   display: block;
125 |   font-size: 13px;
126 |   font-weight: normal;
127 |   color: #777;
128 | }
129 |
130 | .buttons {
131 |   display: flex;
132 |   gap: 12px;
133 |   margin-top: 16px;
134 | }
135 |
136 | button {
137 |   flex: 1;
138 |   border: none;
139 |   border-radius: 8px;
140 |   padding: 15px;
141 |   font-size: 16px;
142 |   font-weight: bold;
143 |   color: white;
144 |   cursor: pointer;
145 | }
146 |
147 | button:disabled {
148 |   opacity: 0.6;
149 | }
150 |
151 | .done {
152 |   background: var(--green);
153 | }
154 |
155 | .fail {
156 |   background: var(--red);
157 | }
158 |
159 | .retry {
160 |   background: var(--navy);
161 |   display: block;
162 |   width: 100%;
163 |   margin-top: 12px;
164 | }
165 |
166 | .result {
167 |   margin-top: 16px;
168 |   padding: 14px;
169 |   border-radius: 8px;
170 |   text-align: center;
171 |   line-height: 1.5;
172 | }
173 |
174 | .result b:first-child {
175 |   display: block;
176 |   font-size: 18px;
177 |   margin-bottom: 4px;
178 | }
179 |
180 | .success {
181 |   background: #e3f5ea;
182 |   color: var(--green);
183 | }
184 |
185 | .failed {
186 |   background: #fdeaea;
187 |   color: var(--red);
188 | }
189 |
190 | .note {
191 |   margin-top: 12px;
192 |   font-size: 13px;
193 |   color: #555;
194 |   text-align: center;
195 | }
```

```
196 |
197 | footer {
198 |   text-align: center;
199 |   font-size: 12px;
200 |   color: #888;
201 | }
```

payment-website/index.html

```
 1 | <!doctype html>
 2 | <html lang="en">
 3 |   <head>
 4 |     <meta charset="UTF-8" />
 5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 6 |     <title>SkyWay Payment</title>
 7 |   </head>
 8 |   <body>
 9 |     <div id="root"></div>
10 |     <script type="module" src="/src/main.jsx"></script>
11 |   </body>
12 | </html>
```

payment-website/vite.config.js

```
 1 | import { defineConfig } from 'vite'
 2 | import react from '@vitejs/plugin-react'
 3 |
 4 | // host: true lets the phone open this website using the laptop's Wi-Fi IP
 5 | export default defineConfig({
 6 |   plugins: [react()],
 7 |   server: {
 8 |     host: true,
 9 |     port: 5173
10 |   }
11 | })
```

payment-website/package.json

```
 1 | {
 2 |   "name": "skyway-payment",
 3 |   "private": true,
 4 |   "version": "1.0.0",
 5 |   "type": "module",
 6 |   "scripts": {
 7 |     "dev": "vite --host",
 8 |     "build": "vite build"
 9 |   },
10 |   "dependencies": {
11 |     "react": "18.3.1",
12 |     "react-dom": "18.3.1"
13 |   },
14 |   "devDependencies": {
15 |     "@vitejs/plugin-react": "4.3.4",
16 |     "vite": "5.4.11"
17 |   }
18 | }
```

run.sh

```
1 | #!/bin/bash
2 | # compile (.java in src -> .class in classes) and run on Mac / Linux
3 | javac -d classes -cp "lib/*" src/*.java && java -cp "classes:lib/*" AirlineMain
```

run.bat

```
@echo off
REM compile (.java in src -> .class in classes) and run on Windows
javac -d classes -cp "lib/*" src\*.java
if errorlevel 1 goto end
java -cp "classes;lib/*" AirlineMain
:end
pause
```

config.properties.example

```
# Gmail settings for sending tickets on email
# Use a Gmail App Password here, not your normal Gmail password
mail.username=your_email@gmail.com
mail.password=your_gmail_app_password
```

.vscode/settings.json

```
{
    "java.project.sourcePaths": ["src"],
    "java.project.outputPath": "classes",
    "java.project.referencedLibraries": ["lib/**/*.jar"]
}
```

.gitignore

```
# Sensitive configuration
config.properties

# Build outputs
classes/
*.class

# Generated PDFs and logs
tickets/*.pdf
*.log

# System files
.DS_Store
Thumbs.db

# Frontend dependencies and builds
payment-website/node_modules/
payment-website/dist/
```

payment-website/.gitignore

```
node_modules
dist
```